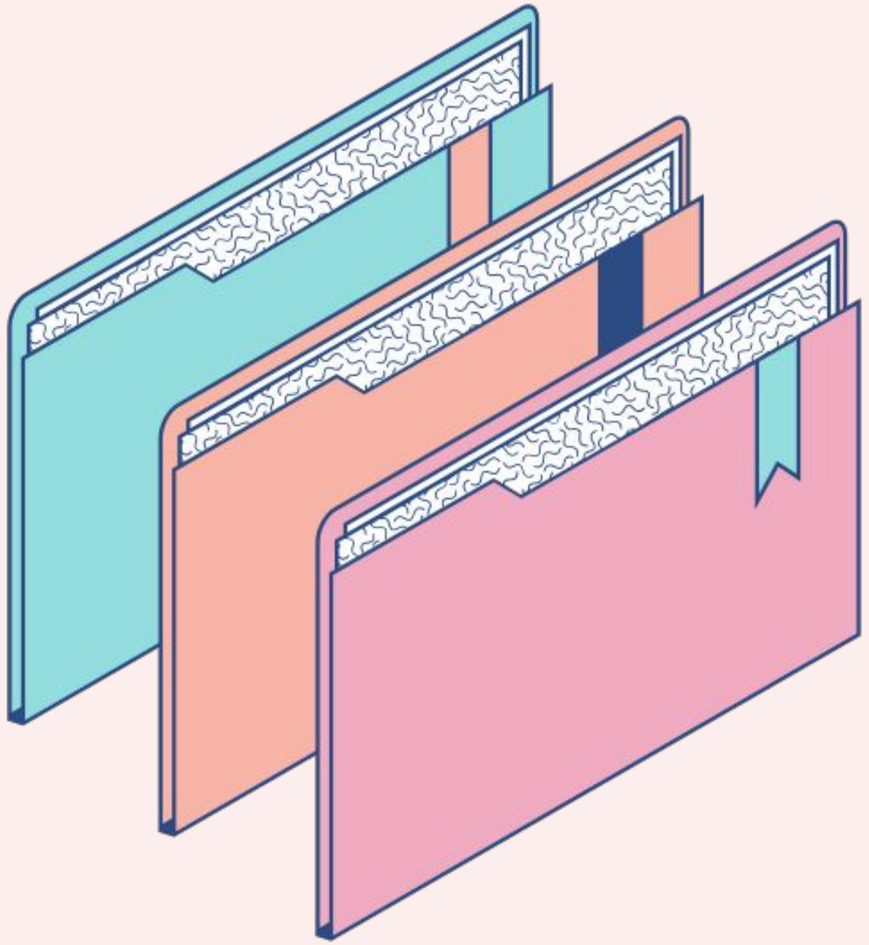


Sistemas Operacionais

III - Processos

Professor

PhD. Eduardo Duarte



Agenda

1. Conceito de Processo
2. Estados do Processo
3. Escalonador de Processos
4. Operações sobre Processos
5. Comunicação Inter processos

Parte I - Conceito de Processo

Estados do Processo



O que é um processo?

Definição de dicionário:

1 Ação & Atividade

Ação continuada, realização contínua e prolongada de alguma atividade; seguimento, curso, decurso.

2 Sequência & Unidade

Sequência contínua de fatos ou operações que apresentam certa unidade ou que se reproduzem com certa regularidade; andamento, desenvolvimento, marcha.



O que é um processo?

Definição para computação:

1 Programa em Execução

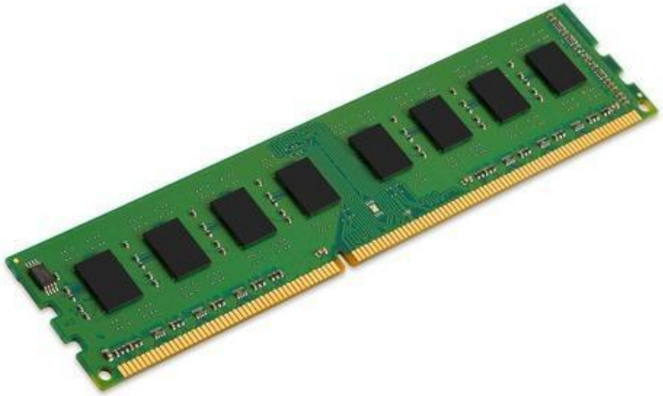
Um processo pode ser considerado simplesmente como um programa em execução no computador.

2 Uso de Recursos

O processo precisará de certos recursos para cumprir sua tarefa. Esses recursos são alocados quando ele é criado ou enquanto está sendo executado.



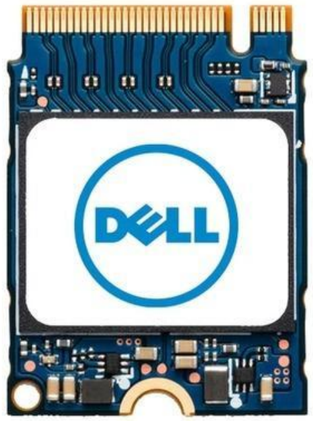
Quais recursos?















Como os processos estão presentes no computador?

O processo é a unidade de trabalho na maioria dos sistemas, que são compostos por um conjunto de processos:

Processos do S.O.

Executam **código de sistema** para gerenciar o hardware e manter a estabilidade.

Processos de Usuário

Executam **código de usuário** associado aos aplicativos iniciados por você.



Um pouco mais sobre processos...

Os primeiros computadores permitiam que apenas **um programa fosse executado de cada vez**.

Esse programa tinha controle total sobre o sistema e acesso a todos os seus recursos, executando de forma linear e isolada.



Reflexão

Qual o problema desse cenário?



Um pouco mais sobre processos...

Multiprogramação Concorrente

Sistemas de computação contemporâneos permitem que vários programas sejam carregados na memória e executados concorrentemente.

A Noção de Processo

Essa evolução demandou controle mais firme e maior compartimentalização dos diversos programas; essas necessidades resultaram na criação do conceito de processo.





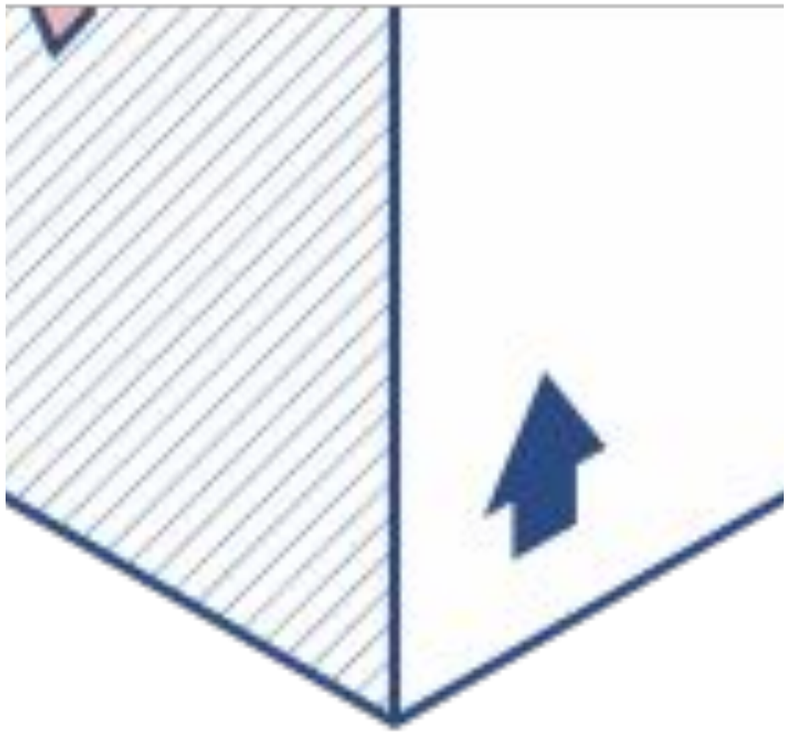
Um pouco mais sobre processos...

Complexidade e Usuários

Quanto mais complexo é o sistema operacional, mais se espera que ele funcione em benefício de seus usuários.

Execução Concorrente

Potencialmente, todos esses processos podem ser executados concorrentemente, com a CPU (ou CPUs) multiplexada entre eles.





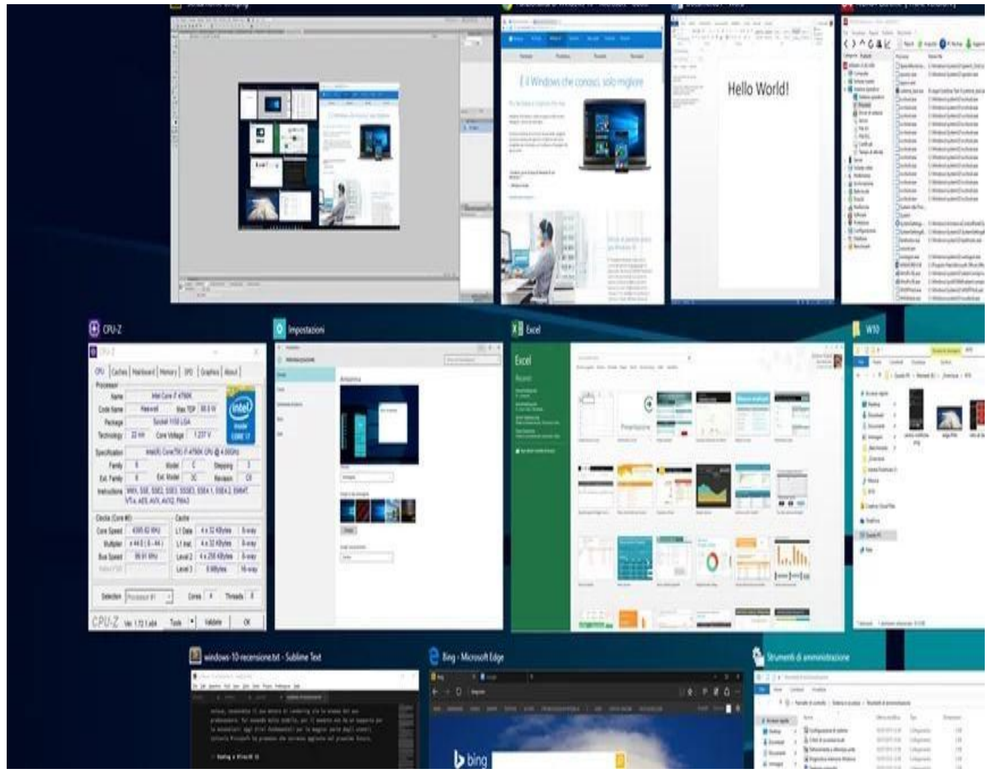
Um pouco mais sobre processos...

Multitarefa Monousuário

Até mesmo em um sistema monousuário, o usuário pode executar vários programas ao mesmo tempo.

Aplicações Simultâneas

Um processador de texto, um navegador web e um pacote de e-mail rodando simultaneamente.





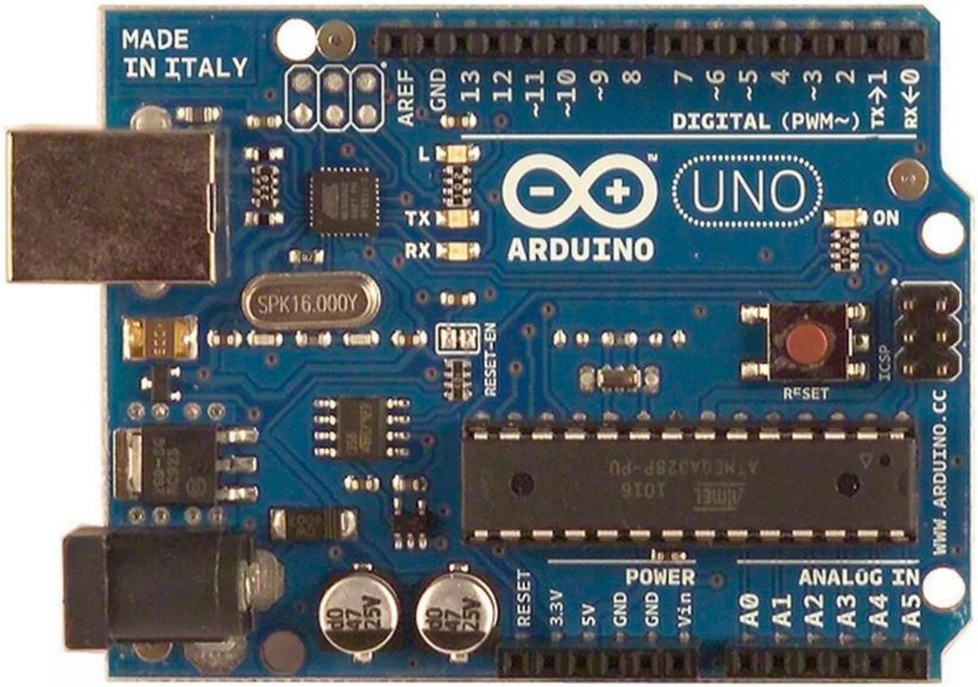
Um pouco mais sobre processos...

Atividades do Sistema

Mesmo que possamos executar apenas um programa de cada vez, sistemas embarcados, por exemplo, que não suporte multitarefa, o S.O deve dar suporte às suas próprias atividades programadas, como o gerenciamento da memória.

Conceito de Processo

Em muitos aspectos, essas atividades são semelhantes e, assim, todas são chamadas de processos.





O processo

Seção de Texto

Um processo é mais do que o código do programa, que também é conhecido como **seção de texto**.

Atividade Corrente

Ele também inclui a **atividade corrente**, conforme representada pelo valor do contador do programa e o conteúdo dos registradores do processador.

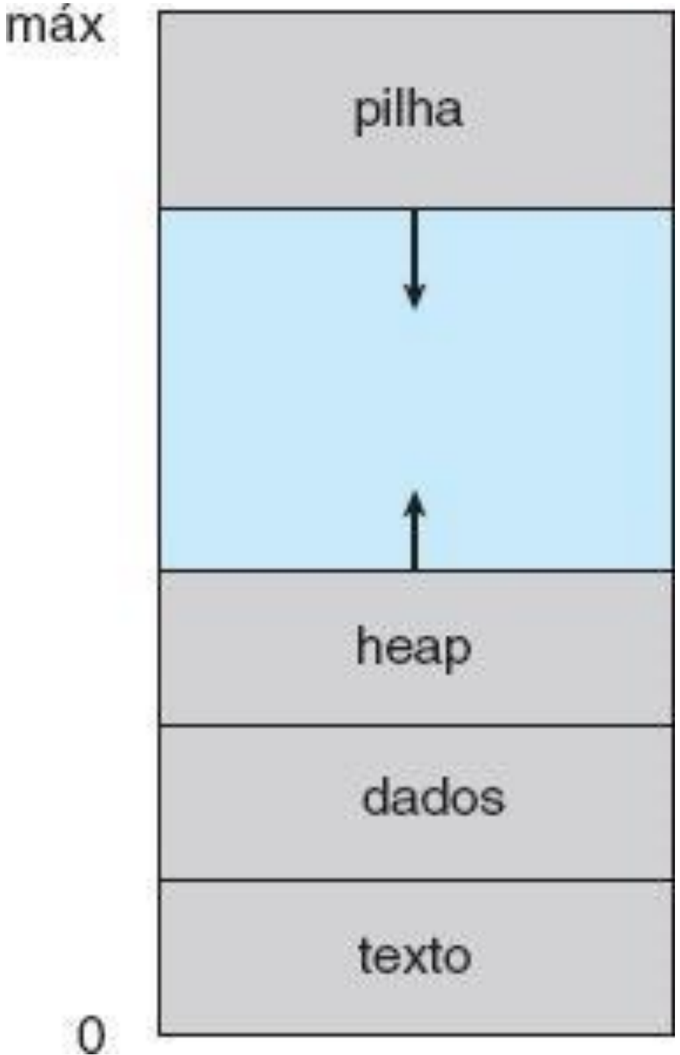


O processo

A Pilha do Processo

Geralmente, um processo também inclui a pilha do processo, que contém dados temporários:

- ➔Parâmetros de funções
- ➔Endereços de retornos
- ➔Variáveis locais





O processo

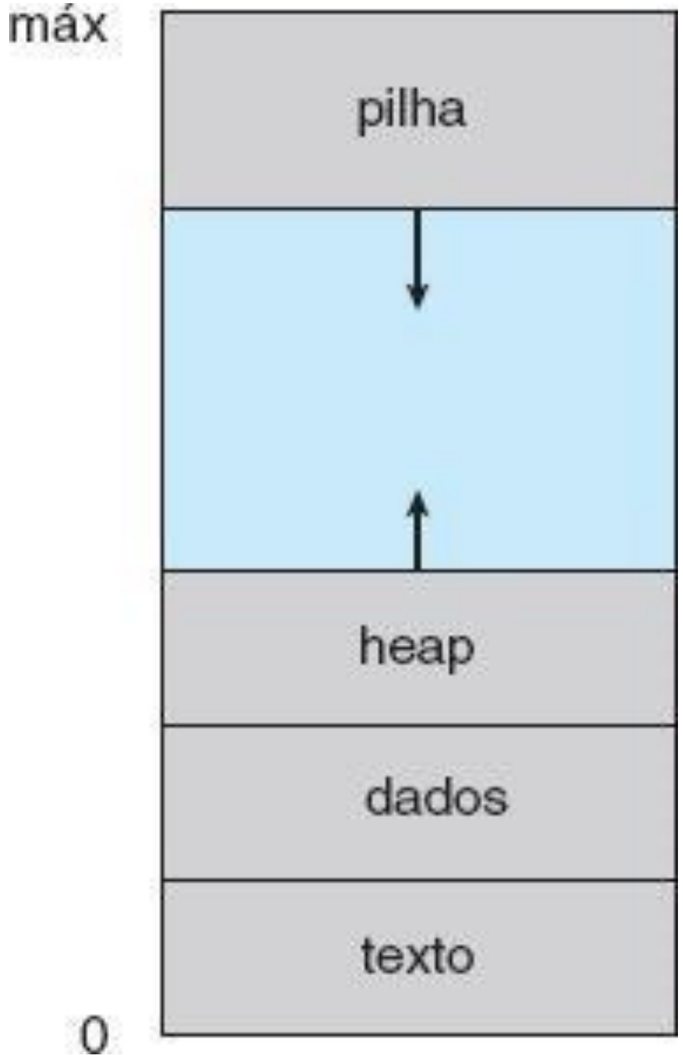
Dados e Heap do Processo

➔ **Seção de Dados:** Contém as variáveis globais e estáticas que ficam acessíveis durante toda a vida do programa.

➔ **Heap (Memória Dinâmica):** Memória reservada sob demanda durante a execução.

O que é Alocação Dinâmica?

Diferente do tamanho fixo, ela funciona como uma **memória flexível**. O programa solicita espaço temporário (ex: criando objetos ou listas de tamanho variável) e depois o libera, garantindo uso eficiente dos recursos do sistema.





O processo

Vale destacar!

Um programa por si só **não é um processo**. Um programa é uma **entidade passiva**, como um arquivo contendo uma lista de instruções armazenadas em disco (geralmente chamado de **arquivo executável**).



O processo

Vale destacar!

Por outro lado, um processo é uma **entidade ativa**, com um **contador de programa** especificando a próxima instrução a ser executada e um conjunto de recursos associados.

Um programa torna-se um processo quando um **arquivo executável** é **carregado na memória**.



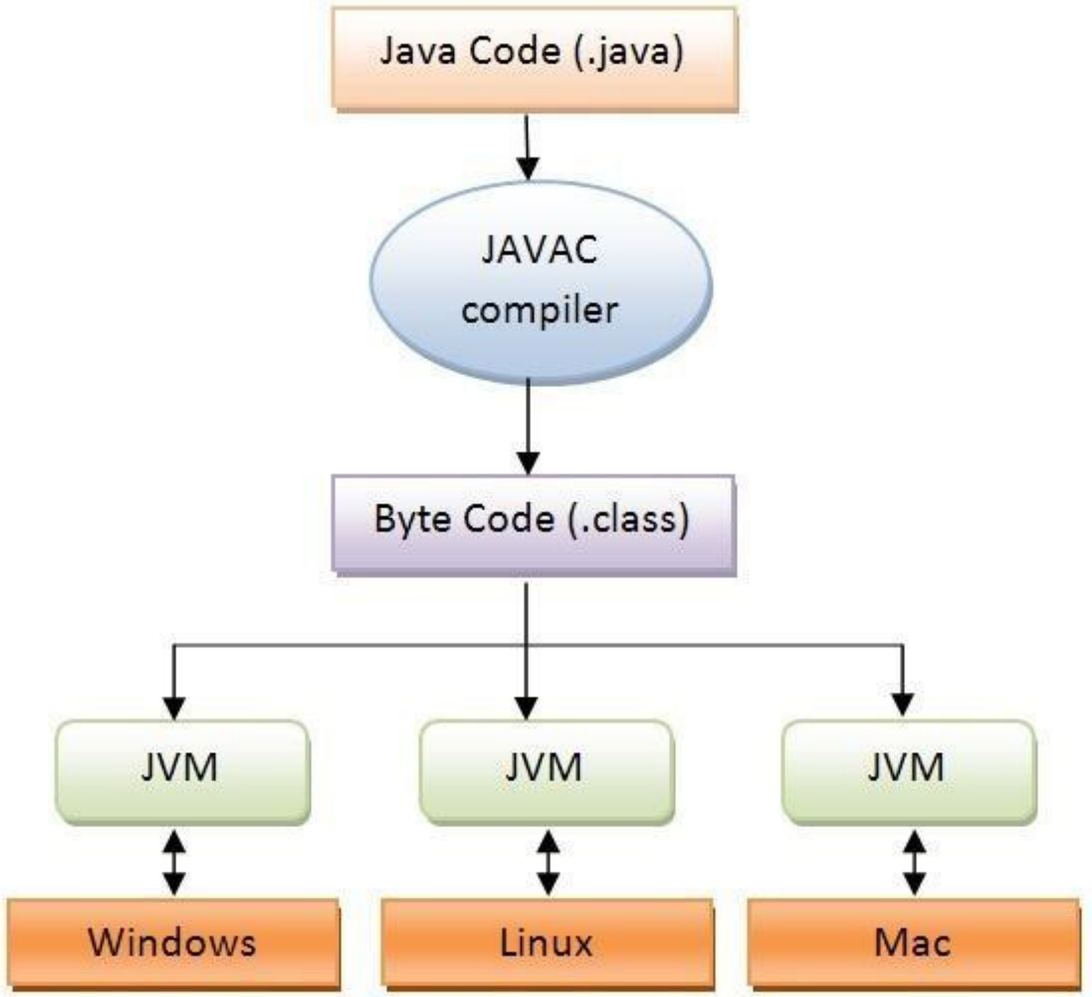
O processo

Ambiente de Execução

O próprio processo pode ser um ambiente de execução para outros códigos. O ambiente de programação Java fornece um bom exemplo.

A Máquina Virtual Java (JVM)

Na maioria dos casos, um programa executável Java é operado dentro da máquina virtual Java (JVM).





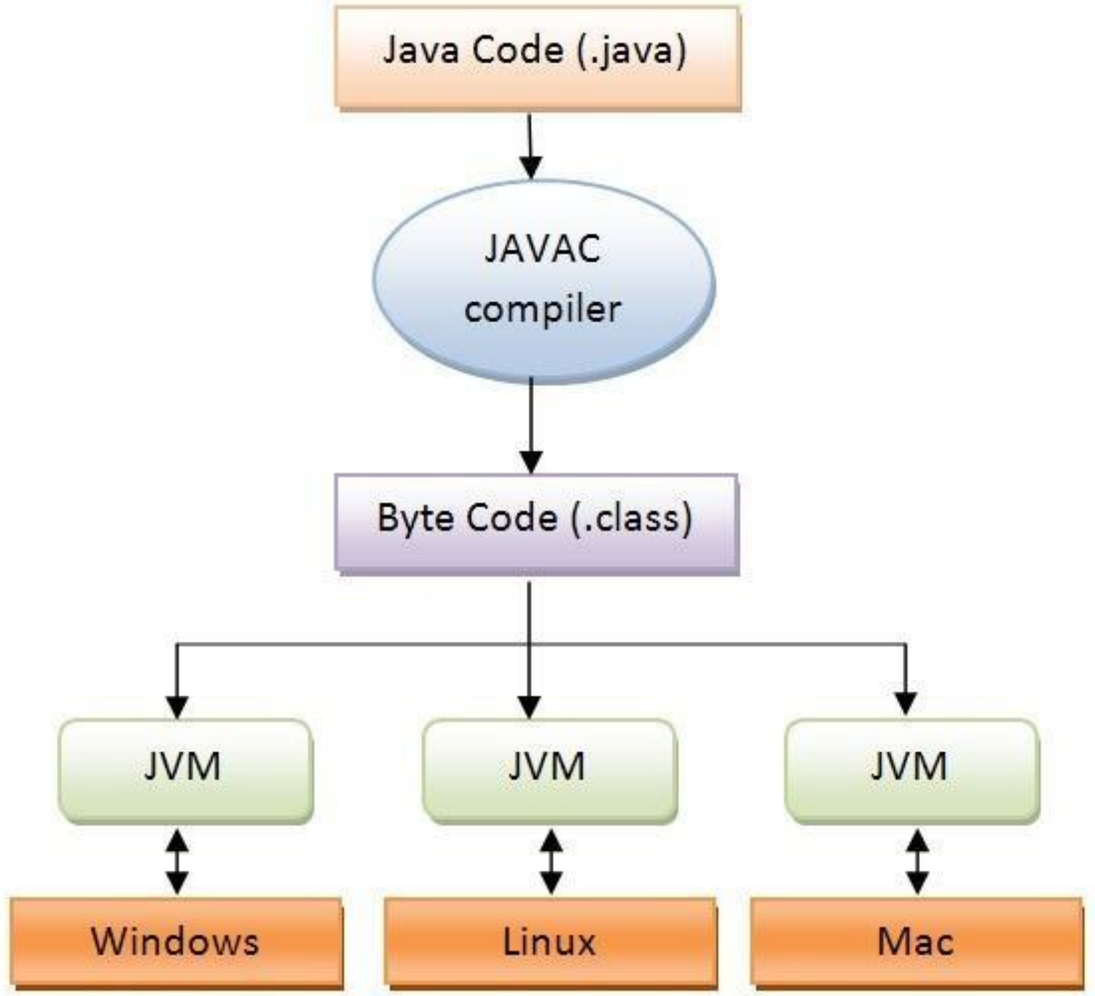
O processo

Execução da JVM

O comando ***java*** executa a JVM como um processo comum que, por sua vez, executa o programa ***Java Program*** na máquina virtual.

Conceito de Simulação

O conceito é o mesmo da simulação, exceto pelo fato de o código ser escrito na linguagem **Java** em vez de ser escrito a partir de um conjunto de instruções diferentes.



Parte II - Estado do processo



O estado do processo

Quando um processo é executado, ele **muda de estado**.

O estado de um processo é definido, em parte, pela **atividade corrente** do processo.

Um processo pode estar em um dos seguintes estados:



O estado do processo

- **Novo**
- **Em execução**
- **Em espera**
- **Pronto**
- **Concluído**



O estado do processo

Novo	O processo está sendo criado.
Em execução	
Em espera	
Pronto	
Concluído	



O estado do processo

Novo	O processo está sendo criado.
Em execução	Instruções estão sendo executadas.
Em espera	
Pronto	
Concluído	



O estado do processo

Novo	O processo está sendo criado.
Em execução	Instruções estão sendo executadas.
Em espera	O processo está esperando que algum evento ocorra (como a conclusão de um I/O ou o recebimento de um sinal).
Pronto	
Concluído	



O estado do processo

Novo	O processo está sendo criado.
Em execução	Instruções estão sendo executadas.
Em espera	O processo está esperando que algum evento ocorra (como a conclusão de um I/O ou o recebimento de um sinal).
Pronto	O processo está esperando que seja atribuído a um processador.
Concluído	



O estado do processo

Novo	O processo está sendo criado.
Em execução	Instruções estão sendo executadas.
Em espera	O processo está esperando que algum evento ocorra (como a conclusão de um I/O ou o recebimento de um sinal).
Pronto	O processo está esperando que seja atribuído a um processador.
Concluído	O processo terminou sua execução.



O estado do processo

É importante saber que apenas **um único processo** pode estar em **execução** em algum processador a cada instante.

Mas muitos processos podem estar **prontos** e em **espera**.





Bloco de Controle de Processo

Cada processo é representado, no sistema operacional, por um **bloco de controle de processo** (PCB – *process control block*) – também chamado **bloco de controle de tarefa**.

Informações Associadas

O bloco contém muitas informações associadas a um processo específico, como mostrado na estrutura ao lado.

Estrutura do PCB
estado do processo
número do processo
contador do programa
registradores
limites da memória
lista de arquivos abertos
...



Bloco de Controle de Processo

Estado do processo

O estado pode ser **novo**, **pronto**, em **execução**, em **espera**, **parado**, e assim por diante.

Contador do programa

O contador indica o **endereço da próxima instrução** a ser executada para esse processo.

Estrutura do PCB
estado do processo
número do processo
contador do programa
registradores
limites da memória
lista de arquivos abertos
...



Bloco de Controle de Processo

Registradores da CPU

O que incluem:

- Acumuladores e registradores de uso geral
- Registradores de índice e ponteiros de pilhas
- Informações do código de condição

Importância do Salvamento:

Essas informações de estado devem ser salvas obrigatoriamente quando ocorre uma interrupção (junto com o contador do programa), permitindo que o processo seja retomado exatamente de onde parou mais tarde.

Estrutura do PCB
estado do processo
número do processo
contador do programa
registradores
limites da memória
lista de arquivos abertos
...



Bloco de Controle de Processo

Informações de scheduling da CPU

Essas informações incluem a prioridade de um processo, ponteiros para filas de scheduling e quaisquer outros parâmetros de scheduling.

Estrutura do PCB
estado do processo
número do processo
contador do programa
registradores
limites da memória
lista de arquivos abertos
...



Bloco de Controle de Processo

Informações de gerenciamento da memória

Essas informações podem incluir itens como o valor dos registradores base e limite e as tabelas de páginas, ou as tabelas de segmentos, dependendo do sistema de memória usado pelo sistema operacional.

Estrutura do PCB
estado do processo
número do processo
contador do programa
registradores
limites da memória
lista de arquivos abertos
...



Bloco de Controle de Processo

Informações de Contabilização

Essas informações incluem o montante de tempo real e de CPU usados, limites de tempo, números de conta, números de jobs ou processos, e assim por diante.

Informações de Status de I/O

Essas informações incluem a lista de dispositivos de I/O alocados ao processo, uma lista de arquivos abertos, e assim por diante.

Estrutura do PCB
estado do processo
número do processo
contador do programa
registradores
limites da memória
lista de arquivos abertos
...



Threads

Modelo de Processo Único

Thread Único de Controle

- O modelo tradicional sugere que um processo executa apenas uma tarefa por vez.
- Um programa em execução é mapeado diretamente para uma única sequência de instruções.

Exemplo Prático

Processador de Texto

- Ao utilizar um editor como o **Microsoft Word**, o fluxo de execução é linear.
- Um único thread de instruções gerencia todas as ações sequencialmente.



Threads

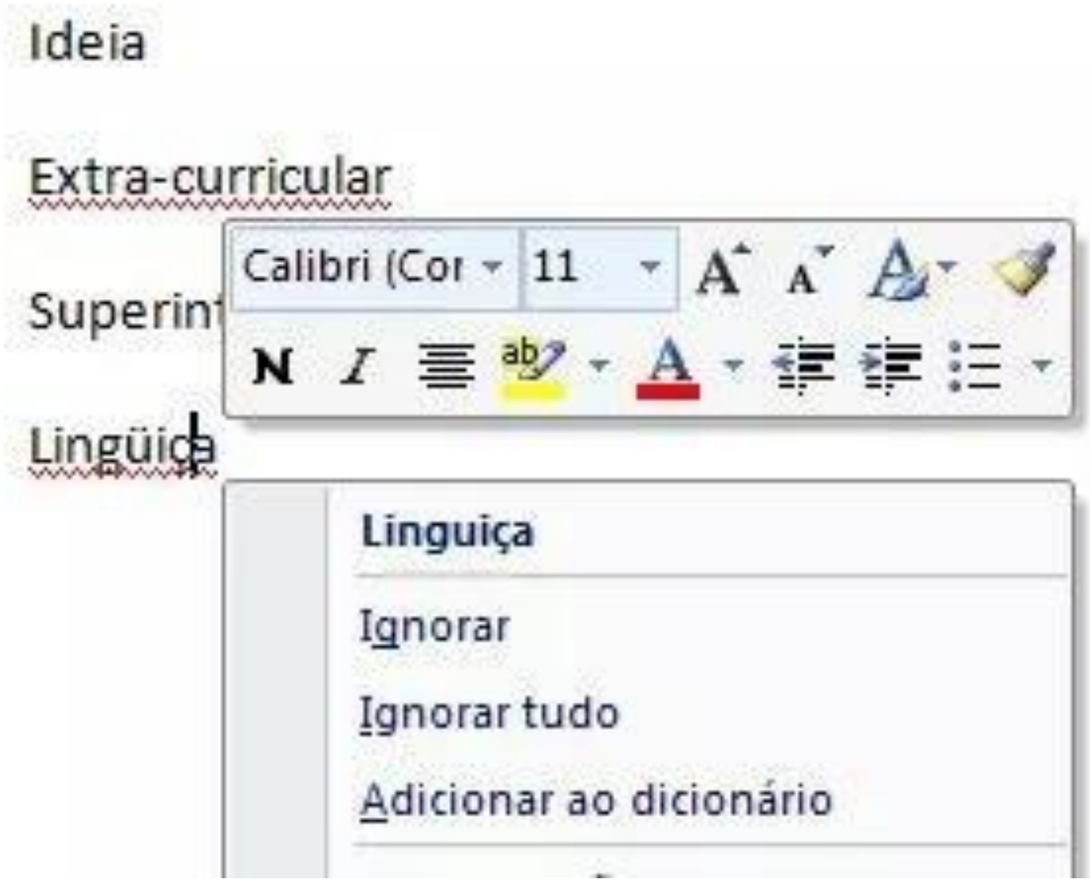
Limitação de Thread Único

Execução Linear

Esse thread de controle único permite que o processo execute apenas uma tarefa de cada vez.

Consequência Prática

O usuário não pode digitar caracteres e executar, simultaneamente, o corretor ortográfico dentro do mesmo processo.





Threads

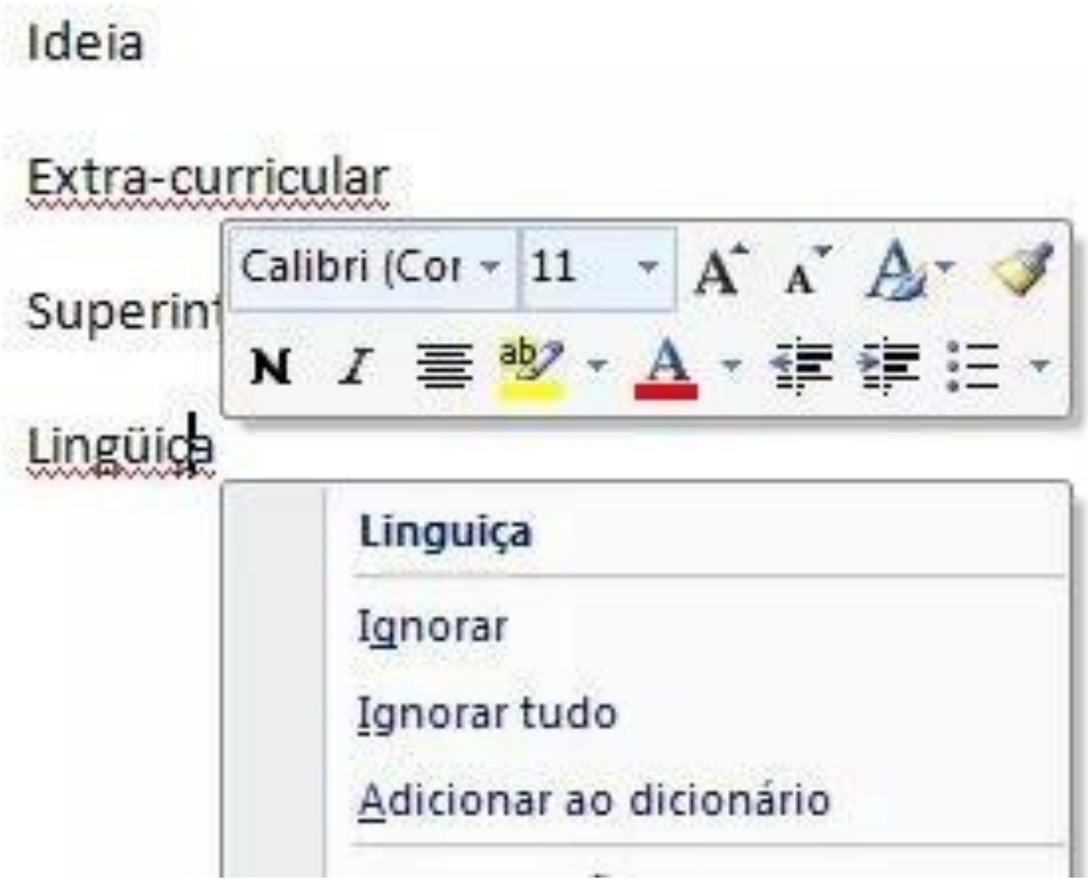
Múltiplos Threads de Execução

Evolução dos Sistemas

• A maioria dos sistemas operacionais modernos estendeu o conceito de processo tradicional.

Suporte a Multitarefa

• Permite que um único processo possua múltiplos threads e desempenhe mais de uma tarefa por vez.





Threads

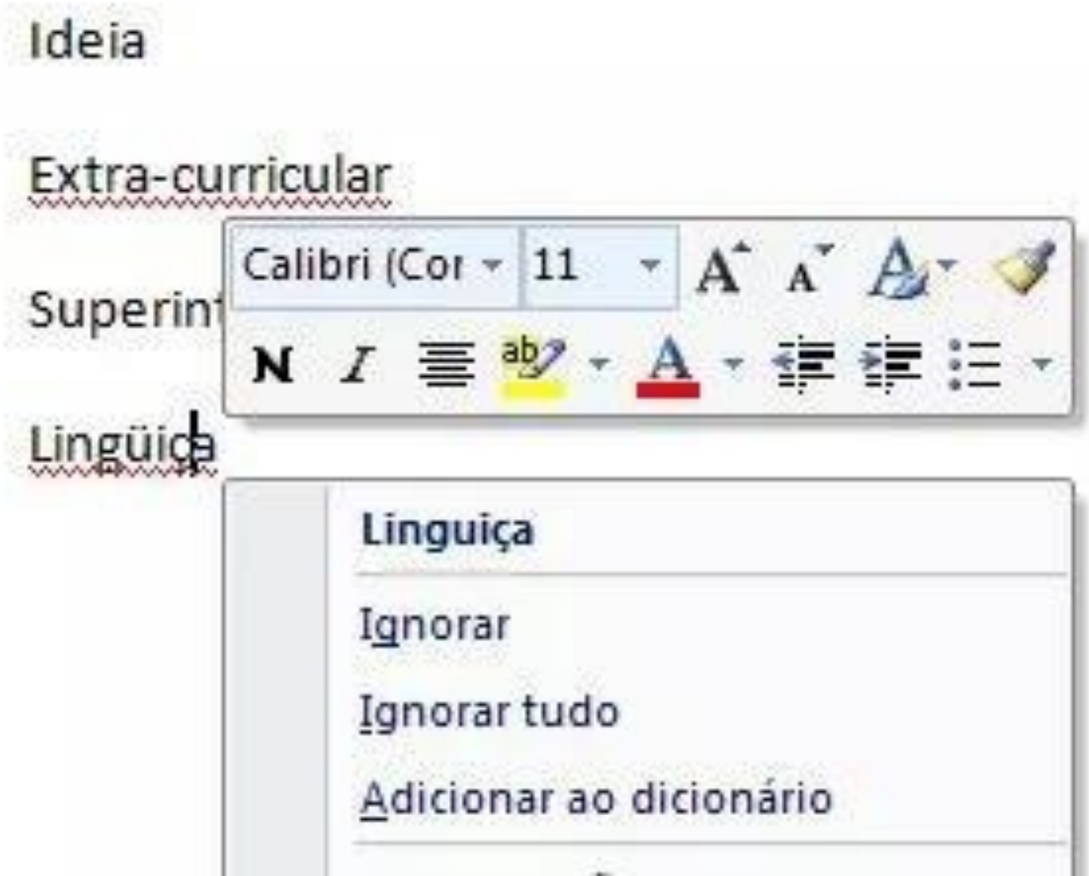
Benefícios & Estrutura

Sistemas Multicore

Esse recurso é particularmente benéfico em sistemas multicore, em que múltiplos threads podem ser executados em paralelo.

Expansão do PCB

Em um sistema que suporte threads, o PCB é expandido de modo a incluir informações para cada thread.



Parte III - Scheduling (escalonamento) de Processos



O que é um Scheduler (escalonador) de processo?

Multiprogramação

Objetivo Principal:

Haver sempre algum processo em execução para maximizar a utilização da CPU.

Compartilhamento de Tempo

Objetivo Principal:

Alternar a CPU entre os processos com tanta frequência que os usuários possam interagir com cada programa enquanto ele é executado.



O que é um Scheduler (escalonador) de processo?

Função do Scheduler

Seleção de Processos:

Para alcançar os objetivos do sistema, o scheduler seleciona um processo disponível (dentre vários em espera) para ser executado diretamente na CPU.

Limitação Física

Processador Único:

Em sistemas monoprocessadores, o limite é estrito: nunca haverá mais de um processo rodando simultaneamente. Apenas um processo ocupa a CPU por vez.



Fila de Scheduling

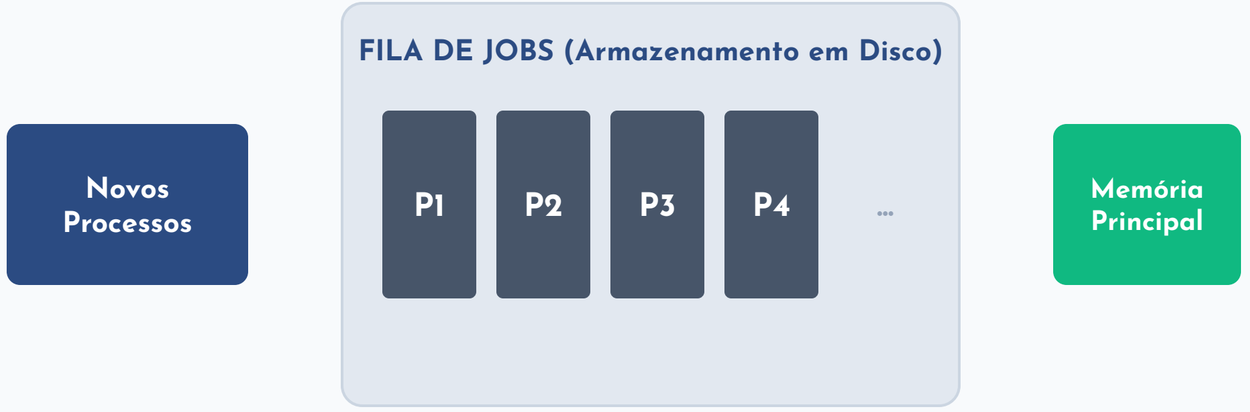
Fila de Jobs

Ingresso no Sistema:

Quando os processos entram no sistema, eles são inseridos na **fila de jobs**.

Esta fila é composta por **todos os processos** presentes no sistema.

DIAGRAMA: FLUXO DE INGRESSO NA FILA DE JOBS





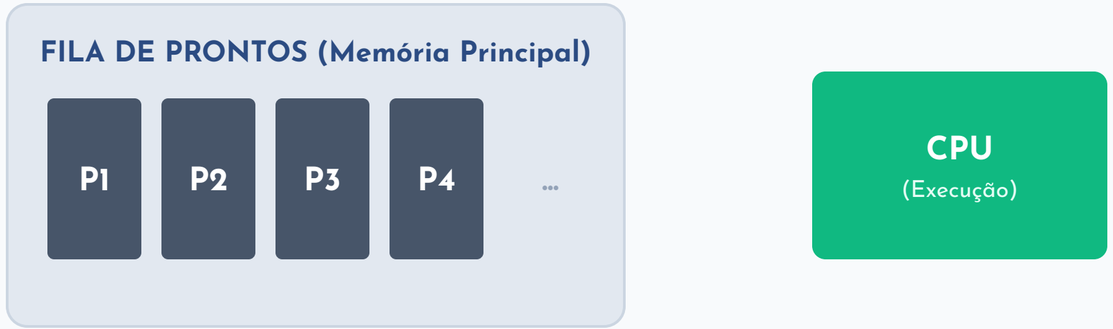
Fila de Scheduling

Fila de Prontos (Ready Queue)

Os processos que estão residindo na **memória principal**, prontos e esperando execução, são mantidos aqui.

Essa estrutura gerencia a transição segura dos processos para a CPU.

DIAGRAMA: FLUXO DA FILA DE PRONTOS PARA A CPU





Fila de Scheduling

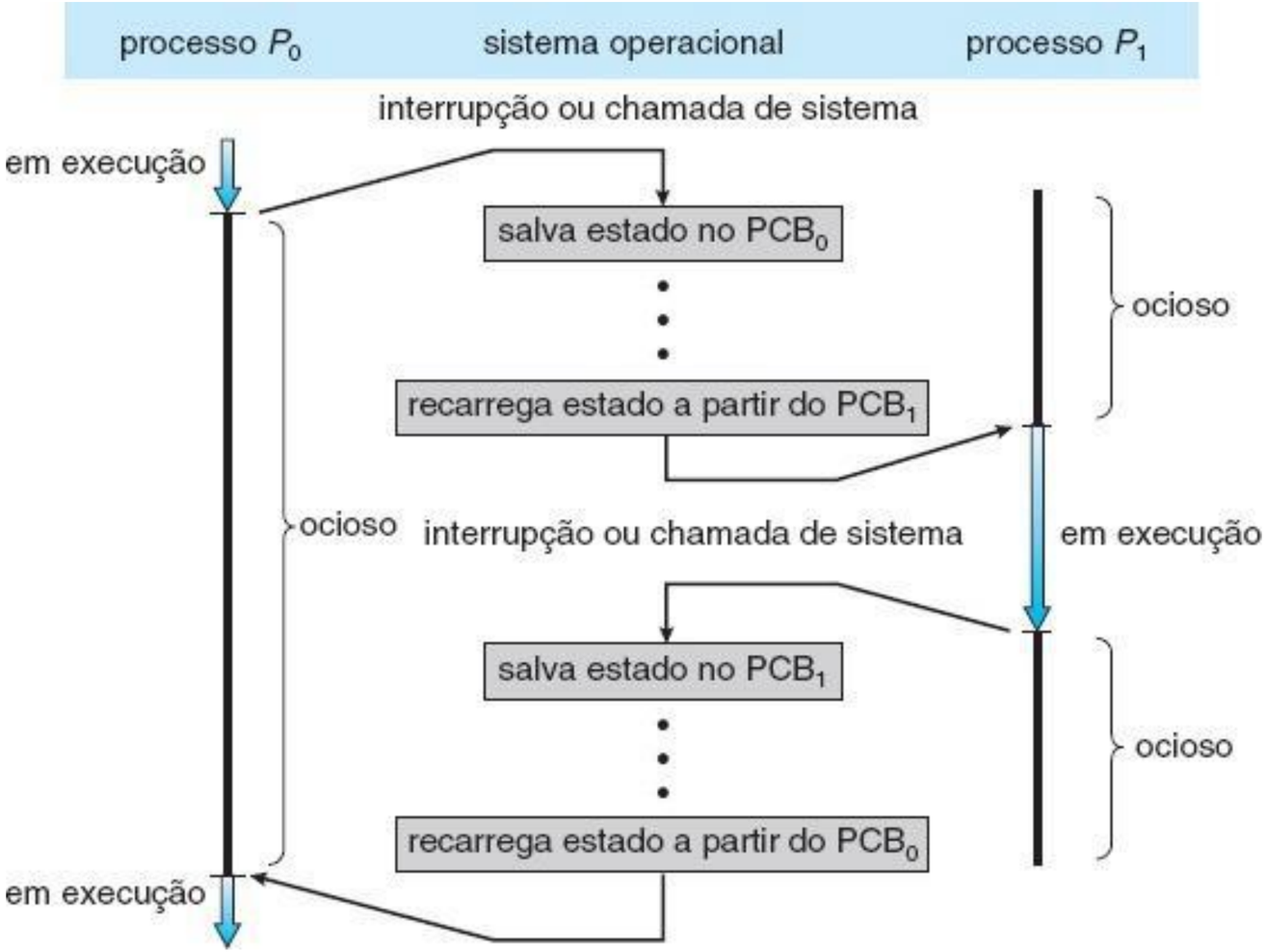
Estrutura de Fila Encadeada

Como funciona a organização?

Lista Encadeada: A fila é armazenada de forma dinâmica como uma lista encadeada de blocos de controle de processos.

Cabeçalho: Contém ponteiros diretos para o **primeiro** e o **último** PCB da fila.

Ponteiro do PCB: Cada PCB possui um campo de ponteiro que aponta para o próximo PCB da fila.





Fila de Scheduling

1. Ciclo de CPU e Interrupção

Quando a **CPU** é alocada a um processo, ele executa por algum tempo e, eventualmente:

- **Para ou é interrompido** para dar espaço a outros processos.
- Entra em estado de espera por um **evento externo** (como a conclusão de I/O).

2. Gargalo de I/O (Entrada/Saída)

Se o processo solicita um **dispositivo compartilhado** como um disco:

- O dispositivo pode estar **ocupado** atendendo a outra requisição.
- O processo deve aguardar a sua vez em uma **fila de dispositivos** dedicada.

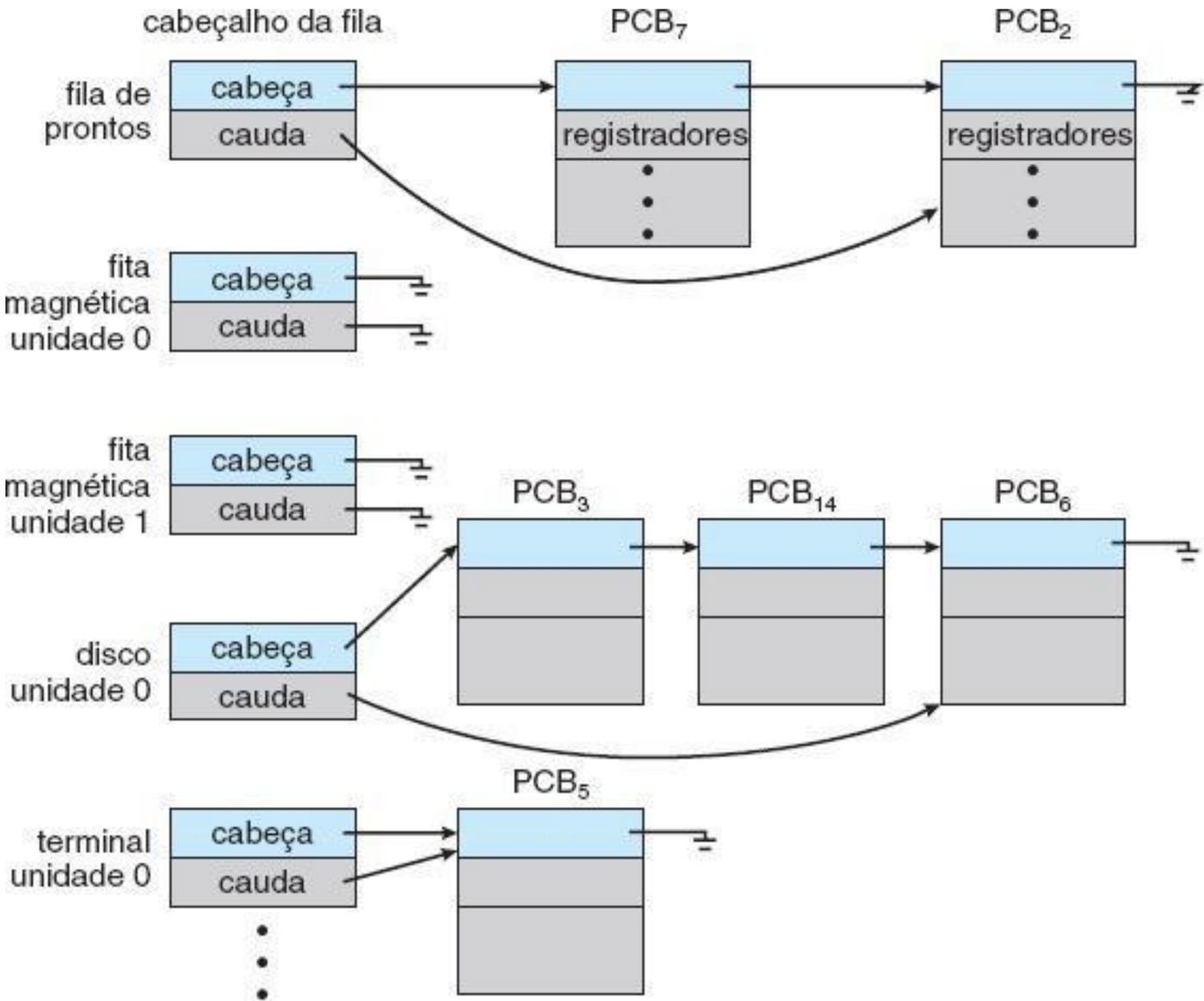


Fila de Scheduling

Fila de Dispositivos (I/O)

Quando um processo solicita entrada ou saída, ele pode não ser atendido de forma imediata:

- **Aguardando o Disco:** O processo pode ter que esperar se o recurso de armazenamento estiver ocupado.
- **Fila do Dispositivo:** É a lista de processos em espera por um dispositivo de E/S específico.
- **Filas Dedicadas:** Cada dispositivo físico do sistema possui sua própria fila exclusiva.

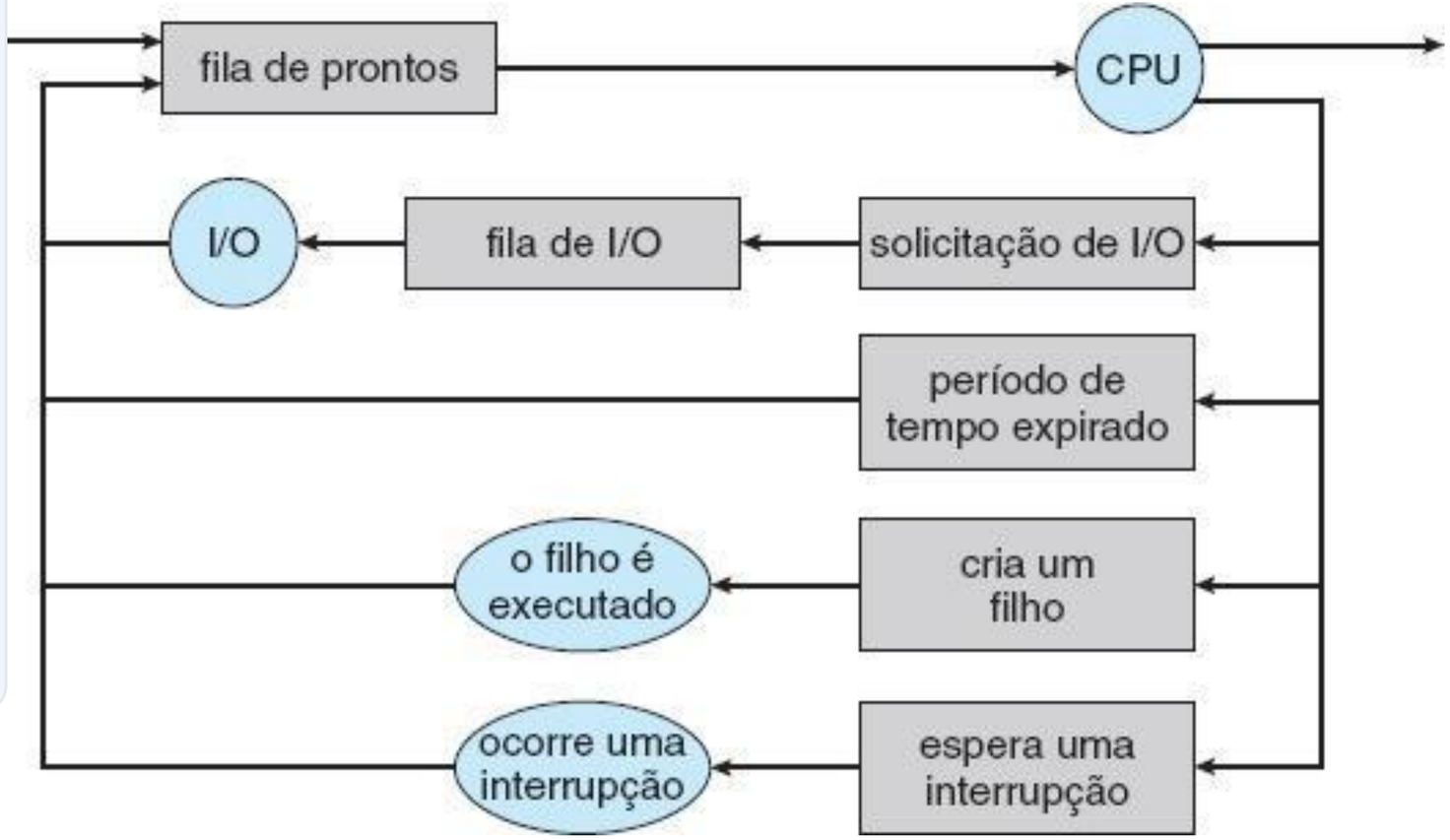




Fila de Scheduling

Estrutura do Fluxo:

- **Filas (Retângulos):** Representam a fila de prontos e as filas de dispositivos de E/S.
- **Recursos (Círculos):** Dispositivos e CPU que servem ativamente às respectivas filas.
- **Fluxo (Setas):** Indicam como os processos transitam e se comportam no sistema.



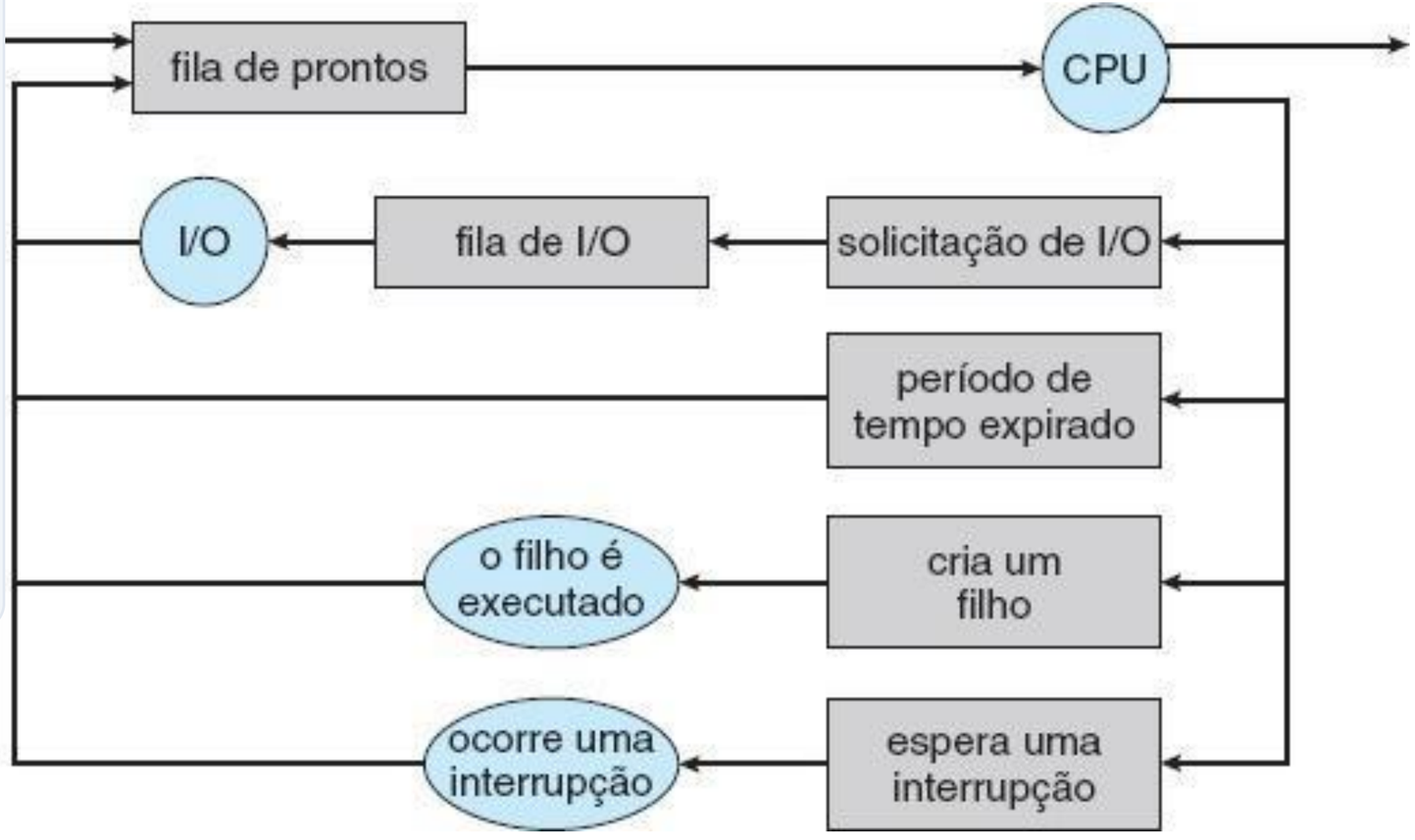


Fila de Scheduling

Fluxo de Processos:

Inicialmente, um novo processo é inserido na **fila de prontos**. Ele aguarda até ser selecionado para execução, ou despachado.

Uma vez que a **CPU** seja alocada ao processo e este entre em execução, um entre vários eventos pode ocorrer:





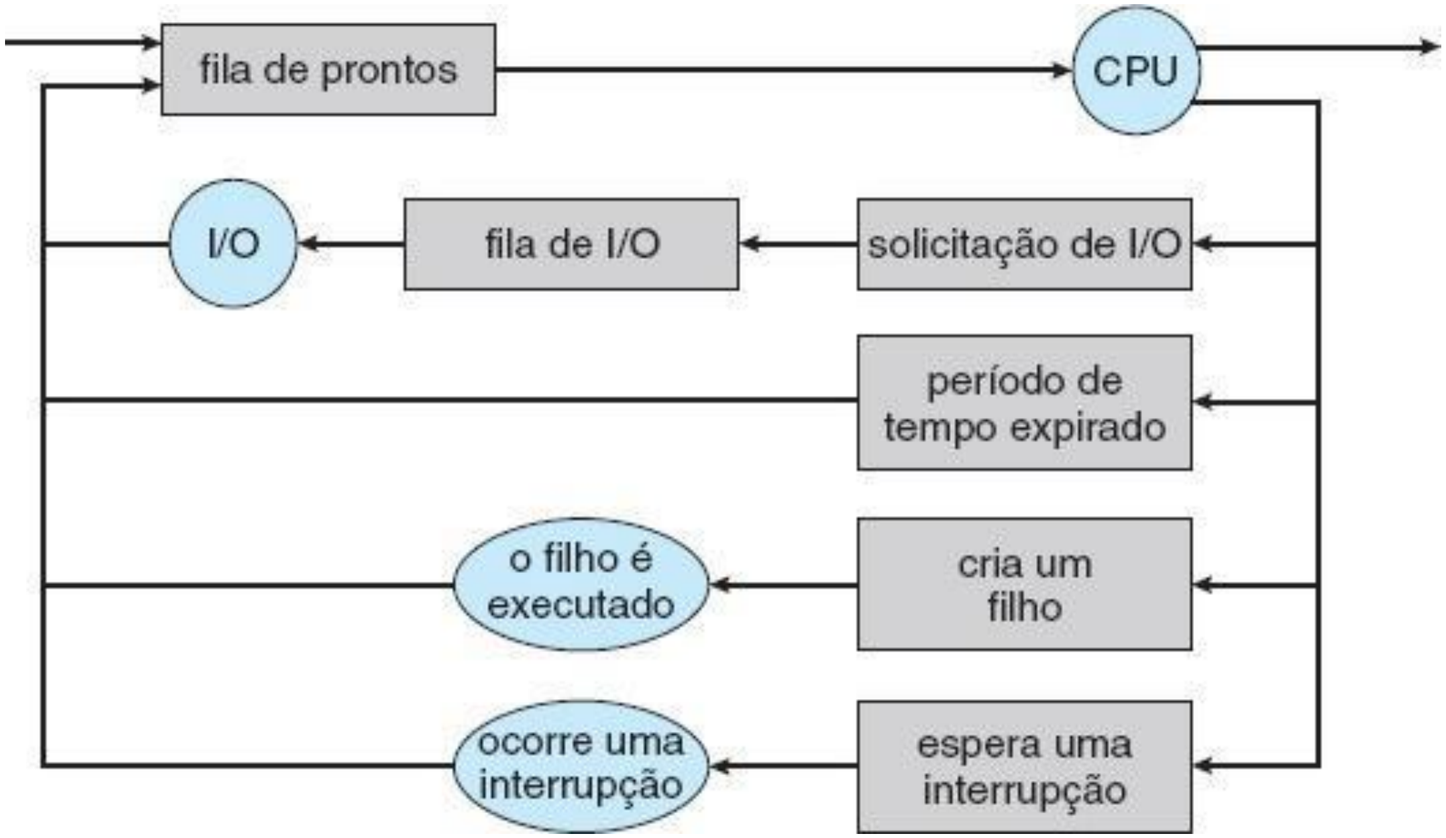
Fila de Scheduling

Ações do Processo na CPU:

1. Solicitação de E/S: O processo pode emitir uma solicitação de I/O e ser inserido em uma fila de E/S para aguardar o dispositivo.

2. Criação de Filho: O processo pode criar um novo processo-filho e aguardar ativamente pelo seu término.

3. Interrupção / Expiração: O processo pode sofrer preempção ou interrupção, sendo removido à força da CPU e devolvido à fila de prontos.



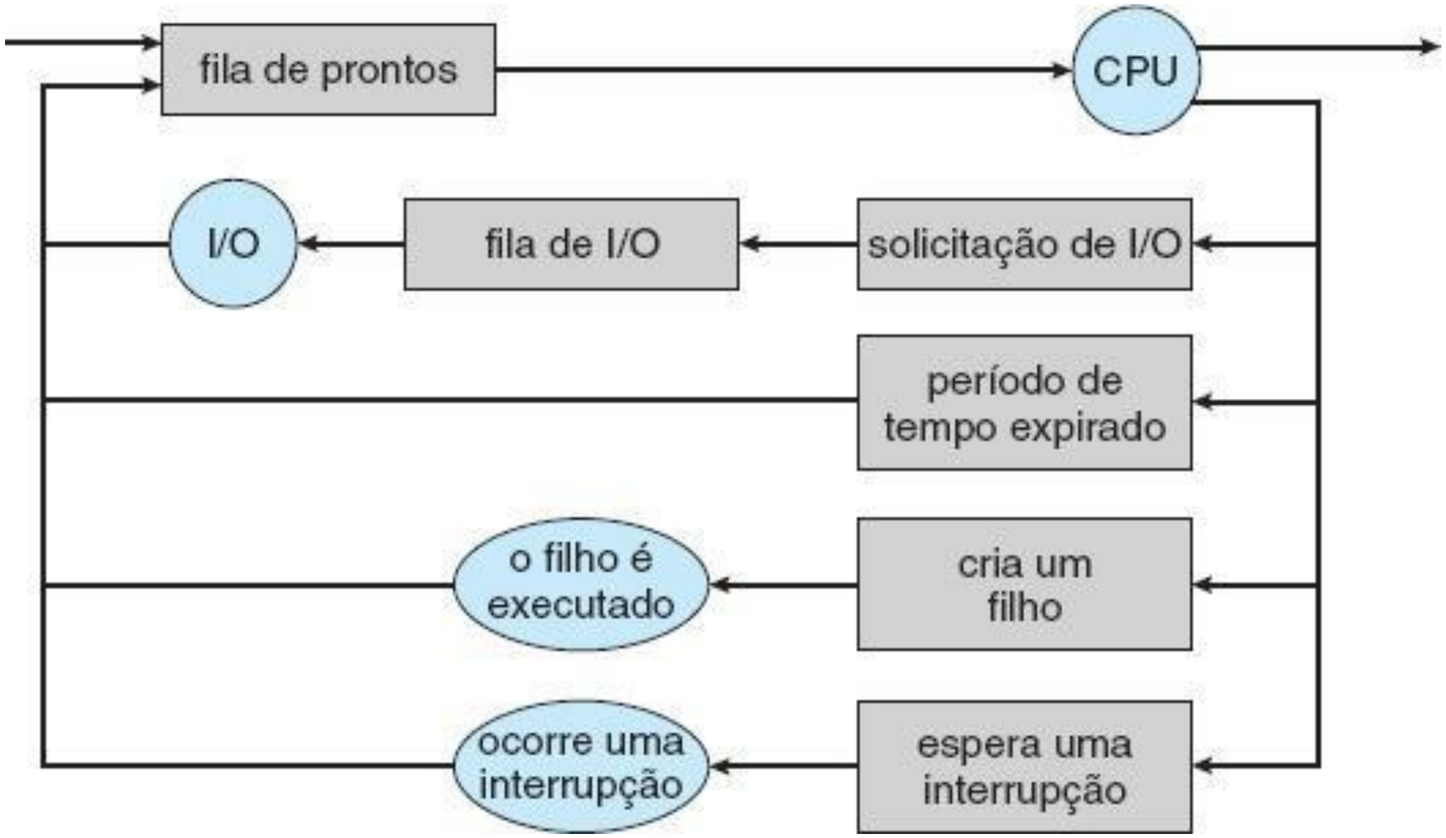


Fila de Scheduling

Retorno e Finalização:

Nos dois primeiros casos, o processo eventualmente passa do **estado de espera** para o **estado de pronto** e é, então, devolvido à **fila de prontos**.

Um processo continua esse ciclo até terminar, momento em que é **removido de todas as filas** e tem seu **PCB** e recursos desalocados.





Schedulers

Fila de Scheduling

Durante o seu ciclo de vida, um processo transita por diversas filas de scheduling.

Seleção: O sistema operacional deve escolher, de forma estruturada, quais processos dessas filas serão executados a seguir. Essa importante tarefa de decisão é realizada pelo **scheduler adequado**.

Sistemas em Lote (Batch)

Em sistemas do tipo batch, a quantidade de processos submetidos frequentemente excede a capacidade de processamento imediato.

Armazenamento: Esses novos processos são mantidos temporariamente em um dispositivo de armazenamento de massa (como um **disco rígido**) para que possam ser executados posteriormente.



Schedulers

Longo Prazo

Ou *Job Scheduler*, define quais programas entram no sistema para serem processados.

Fluxo de Execução:

DISCO ➔ MEMÓRIA

- Controla o grau de multiprogramação
- Executado com menor frequência

Curto Prazo

Ou *CPU Scheduler*, seleciona qual processo pronto na memória deve receber a CPU agora.

Fluxo de Execução:

MEMÓRIA ➔ CPU

- Altíssima frequência de execução
- Deve ser extremamente rápido



Schedulers

A principal diferença entre os schedulers está na **frequência de execução**.

Frequência Extrema

O scheduler de curto prazo (CPU) opera em intervalos de tempo incrivelmente pequenos.

Execução Recomendada:

1 Vez a cada 100ms

Garante a rápida distribuição da capacidade de processamento.

Decisão Veloz

Seleciona com precisão o novo processo para ocupar o processador imediatamente.

Tempo de Execução:

Apenas Milissegundos

Rápida transição antes de entrar em espera por uma solicitação de I/O.



Schedulers

Alta Frequência

Como o tempo entre as execuções é extremamente curto, o scheduler de curto prazo precisa agir quase instantaneamente.

Qualquer atraso na tomada de decisão consome ciclos valiosos que deveriam ser dedicados aos processos.

Impacto do Overhead

Se a decisão do scheduler leva **10 ms** para executar um processo por **100 ms**:

9% de Desperdício

`10 / (100 + 10) = 9%` da capacidade da CPU perdida apenas com a organização do trabalho.



Schedulers

Frequência Menor

O scheduler de longo prazo é executado com muito menos frequência.

Intervalo:

Minutos

Podem separar a criação de um novo processo e o seguinte.

Multiprogramação

Controla diretamente o grau de multiprogramação do sistema.

Objetivo:

Processos na Memória

Determina quantos processos residem de fato na RAM.

Fluxo Estável

Mantém a estabilidade do fluxo de trabalho no sistema operacional.

Equação de Equilíbrio:

Criação = Saída

A taxa de novos processos deve ser igual à dos que saem.



Schedulers

Seleção Cuidadosa

O scheduler de longo prazo deve fazer uma escolha criteriosa dos processos que entram no sistema.

Isso evita sobrecarga e garante o uso eficiente de todos os recursos.

Mix de Processos

É essencial obter um bom equilíbrio na fila de execução.

Se todos os processos forem do mesmo tipo, haverá gargalos em alguns recursos e ociosidade em outros.

I/O vs CPU Bound

A maioria dos processos se divide em:

- **I/O-bound:** passam mais tempo fazendo entrada/saída.
- **CPU-bound:** passam mais tempo executando cálculos.



Schedulers de médio prazo

Sistemas de tempo compartilhado introduzem um nível de agendamento intermediário para remover temporariamente processos da memória, otimizando o fluxo geral.

1. Swapping-Out

Remove os processos ativos da memória RAM e os transfere para o disco.

Alivia a Memória

2. Estado Suspenso

O processo aguarda no disco (área de swap) sem consumir espaço na memória ativa.

Reduz Multiprogramação

3. Swapping-In

Traz o processo de volta do disco para a RAM quando há espaço disponível disponível.

Retoma a Execução



Schedulers de médio prazo

A ideia-chave por trás de um scheduler de médio prazo é que, às vezes, pode ser vantajoso remover processos da memória para otimizar o sistema:

1. Alívio de Memória (Swap-out)

Remove temporariamente processos ativos da memória RAM e os transfere para o disco (área de swap).

- **Reduz a disputa ativa pela CPU e libera espaço na memória principal.**

2. Grau de Multiprogramação

Ao controlar a quantidade de processos em execução concorrente, o sistema operacional evita sobrecarga extrema.

- **Evita o "thrashing" (paginação excessiva) e melhora a vazão geral de tarefas.**



Schedulers de médio prazo

3. Retomada (Swap-in)

Posteriormente, o processo suspenso pode ser reintroduzido na memória física (RAM).

- **Sua execução é retomada exatamente de onde parou.**
- **Essencial para balancear e otimizar o mix de processos ativos.**

O Conceito de Swapping

O esquema completo de mover processos entre a memória principal (RAM) e o disco é chamado de **swapping**.

- **Libera memória física sob demanda.**
- **Garante que novos processos possam ser iniciados.**



Mudança de Contexto

As interrupções forçam o sistema operacional a alternar tarefas na CPU de forma estruturada:

1. Salvar o Contexto

O sistema operacional suspende o processo corrente para executar uma rotina do kernel.

- **Salva o estado atual (registradores e CPU) no respectivo PCB do processo.**

2. Restaurar o Contexto

Após a interrupção ou decisão do escalonador, o sistema operacional recupera a tarefa.

- **Restaura os registradores salvos e retoma a execução exatamente de onde parou.**



Mudança de Contexto

O PCB & Contexto

O contexto é a imagem ativa do processo, mapeada e armazenada diretamente no seu **PCB** (Process Control Block).

- **Registradores da CPU**
- **Estado atual do processo**
- **Gerência de memória**

1. Salvar Estado

Antes de alternar para outra tarefa, o SO precisa suspender a atual e salvar seu estado para evitar perda de dados.

- **Executado em kernel ou usuário**
- **Preserva registradores na CPU**

2. Restaurar Estado

Para reativar um processo, o sistema operacional localiza seu PCB e recarrega as informações anteriormente salvas.

- **Restaura valores nos registradores**
- **Retoma do ponto exato onde parou**



Mudança de Contexto

O Conceito

A alocação da CPU a outro processo requer a execução do salvamento do estado corrente e a restauração de um processo diferente.

- **Alternância de Tarefas**

Como Funciona

O kernel salva o contexto do processo antigo em seu PCB (Process Control Block) e carrega o novo contexto indicado no escalonador.

- **PCB como Repositório**

O Overhead

O tempo gasto na mudança é puro overhead. O sistema operacional não executa trabalho útil para o usuário durante essa troca.

- **Custo de Desempenho**



Multitarefas em Dispositivos Móveis

Em Foreground

• Apenas **uma única aplicação** é executada ativamente em primeiro plano.

• É o aplicativo que está aberto e visível na tela, interagindo diretamente com o usuário.

Em Background (Suspensas)

• Todas as outras aplicações de usuário são **completamente suspensas**.

• Elas permanecem salvas na memória, mas não realizam processamento em segundo plano.



iOS 4: Multitarefa Limitada

Como Funciona

• Permite que uma única aplicação de **foreground** (primeiro plano) seja executada na tela.

• Executa de forma concorrente com múltiplas aplicações específicas em **background** (segundo plano).

Restrições da API

• A API dá suporte à multitarefa de forma **restrita e controlada** pela Apple.

• Disponível apenas para tipos específicos como **reprodução de áudio**, serviços de localização (GPS) e VoIP.



Multitarefas em Dispositivos Móveis

Como Funciona

• O Android **não impõe restrições** rígidas aos tipos de apps em segundo plano.

• Se um app requer processamento ativo, ele deve utilizar um **Serviço**.

• O serviço roda de forma **independente** da interface visual do usuário.

Exemplo Prático

• No app de **streaming de áudio**, ao ser minimizado, a tela é fechada.

• O **serviço de áudio** associado continua ativo em background.

• Ele continua transferindo dados para o hardware para **manter a música tocando**.



Multitarefas em Dispositivos Móveis

Atualmente, tanto **Android** quanto **iOS** dispõem de técnicas para trabalhar em multitarefas com o menor número de restrições possíveis.



Multitarefas: Benefícios das Threads

As técnicas de multitarefa passam pelo uso de **Threads** (abordadas na próxima Unidade):

Capacidade de Resposta

Permite que o app continue executando e respondendo ao usuário mesmo se parte dele estiver bloqueada.

Compartilhar Recursos

Threads compartilham a memória e os recursos do processo pai por padrão, facilitando a comunicação direta.

Economia

Criar threads consome menos tempo e memória do que criar processos inteiros, tornando o sistema mais leve.

Escalabilidade

Permite que threads rodem em paralelo real em múltiplos núcleos de processamento de hardware moderno.

Parte IV - Operações sobre Processos



Operações sobre Processos

Criação

Execução Concorrente

Os sistemas permitem que novos processos sejam criados dinamicamente para executar tarefas em paralelo, dando origem a relações de processos-pai e filhos.

Encerramento

Exclusão Dinâmica

Os processos são excluídos do sistema assim que finalizam suas instruções ou quando requisitado, liberando todos os recursos previamente alocados.

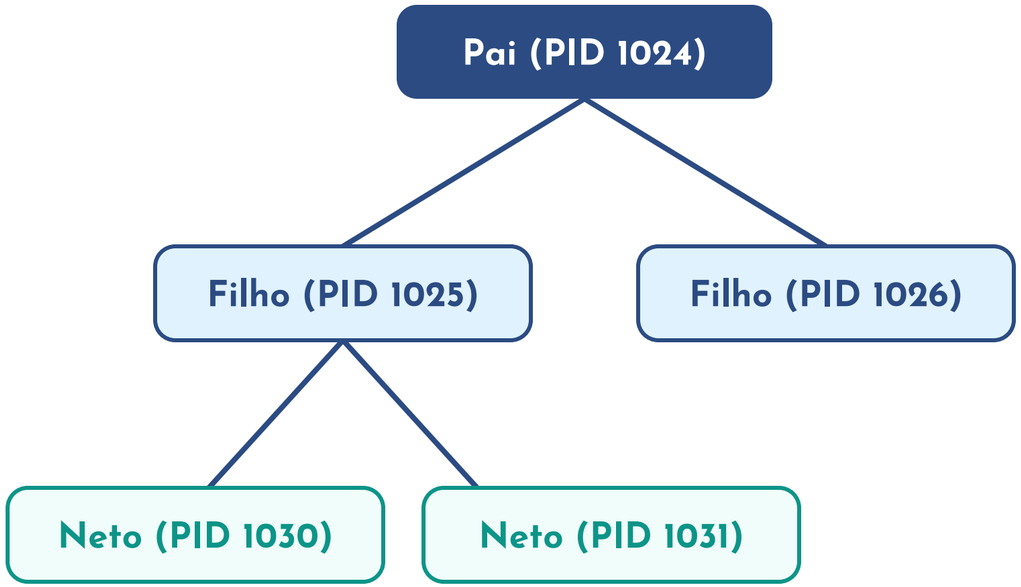


Operações sobre Processos

Relação de Processos

Durante sua execução, um processo pode criar vários processos novos. O processo criador é chamado de **processo-pai**, e os novos são os **processos-filhos**.

Cada novo processo pode, por sua vez, criar outros processos, formando uma estrutura hierárquica conhecida como **árvore de processos**.



Exemplo prático de uma **Árvore de Processos**



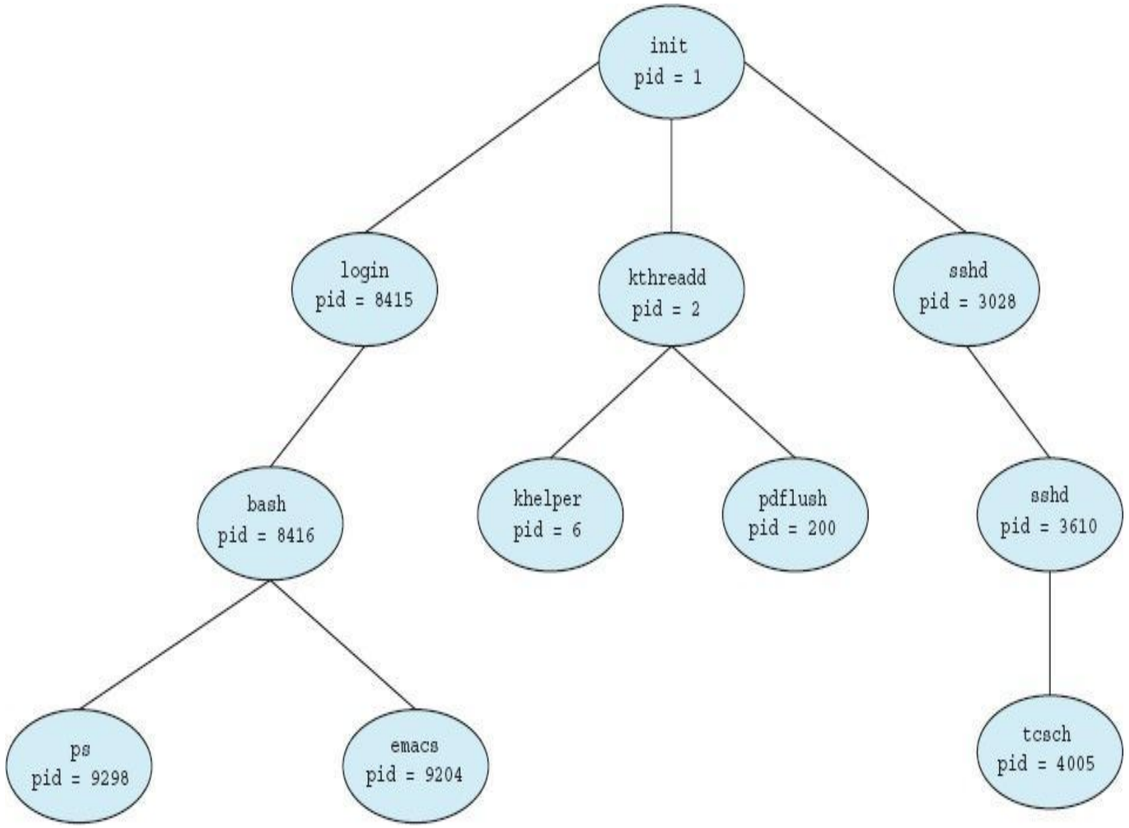
Operações sobre Processos

Identificador único (PID)

Um processo é identificado através de um identificador de processos (ou **PID**) exclusivo, que em geral, é um número inteiro.

Exemplos Práticos:

- **PID = 1:** Processo ***init***, o pai de todos.
- **PID = 2:** Gerenciador de threads do kernel (*kthreadd*).
- **PIDs Dinâmicos:** Atribuídos conforme novos processos são criados.



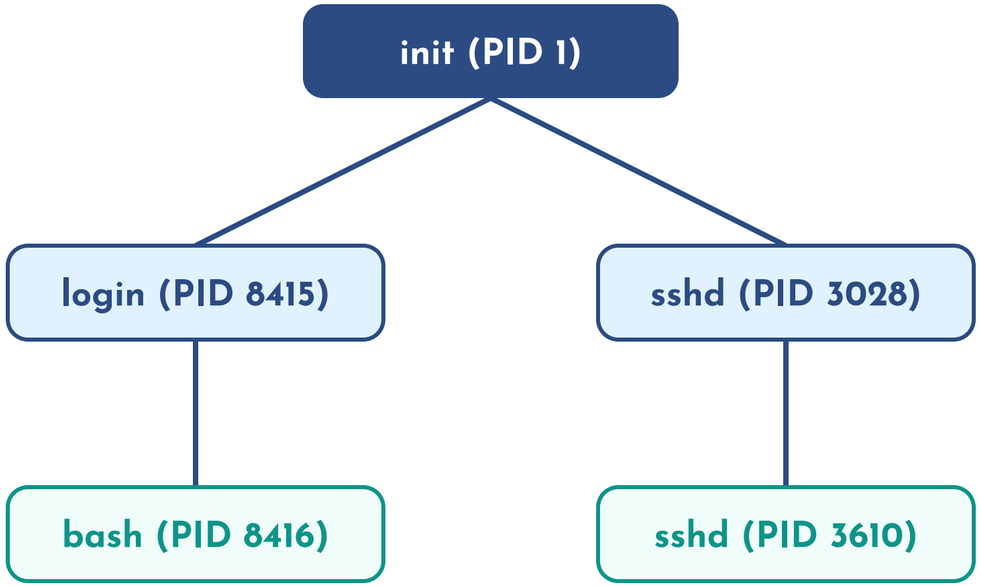
Operações sobre Processos

Identificadores (PID)

Os processos de login e acesso remoto possuem identificadores específicos na hierarquia do sistema operacional.

Exemplos Práticos:

- **PID = 8415:** Processo ***login***, responsável por gerenciar sessões locais.
- **PID = 3028:** Processo ***sshd***, servidor de conexões seguras remotas.
- **PIDs Filhos:** Atribuídos a interpretadores de comandos de usuários conectados.



Relação prática dos processos de **Acesso e Login**



Operações sobre Processos

Necessidade de Recursos

Geralmente, quando um processo cria um processo-filho, este necessita de recursos específicos para executar sua tarefa de forma independente.

Recursos essenciais solicitados:

- **Tempo de CPU** e **Memória RAM**
- **Arquivos** e **Dispositivos de I/O**

Origem e Limitações

A forma como um processo-filho obtém seus recursos define o nível de controle do sistema operacional sobre a execução:

- **Do Sistema Operacional:** Obtém recursos diretamente do S.O., competindo de igual para igual com outros processos.
- **Do Processo-Pai:** Fica estrito a compartilhar ou dividir apenas um subconjunto dos recursos do próprio pai.

Operações sobre Processos

Distribuição de Recursos

O pai pode ter de dividir seus recursos entre seus filhos, ou pode compartilhar alguns recursos entre vários deles.

Recursos Compartilhados:

- **Memória Compartilhada:** Evita duplicação desnecessária de dados.
- **Arquivos Abertos:** Acesso simultâneo a descritores de arquivos.

Controle e Restrição

Restringir um processo-filho a um subconjunto dos recursos do pai impede que ele sobrecarregue o sistema operacional.

Mecanismos de Proteção:

- **Limite de Processos-Filhos:** Impede a criação de processos em cascata infinita.
- **Quotas de Hardware:** Restringe o uso máximo de CPU e memória por subclasse.



Operações sobre Processos

Inicialização de Dados

Além de fornecer diversos recursos físicos e lógicos, o processo-pai pode passar, na inicialização, dados (entradas) ao processo-filho.

Principais Características:

- Envio de parâmetros logo na criação.
- Definição do escopo inicial de trabalho do filho.

Exemplo de Aplicação

Considere um processo cuja função seja exibir o conteúdo de um arquivo na tela de um terminal.

Fluxo do Processo:

- O pai envia o nome do arquivo ***image.jpg*** como entrada.
- O filho recebe esse nome de arquivo para abrir e exibir seu conteúdo.

Fluxo de Abertura de Arquivo

Passo 1

Definição da Entrada

O processo-pai inicia a operação definindo qual arquivo deve ser aberto e processado.

O pai envia o nome do arquivo ***image.jpg*** como o parâmetro inicial.

Passo 2

Abertura e Leitura

O processo-filho é criado e recebe o nome do arquivo enviado pelo pai para execução.

Usando esse nome, o filho localiza, abre o arquivo e realiza a leitura do seu conteúdo original.

Passo 3

Exibição de Saída

O conteúdo lido é direcionado ao dispositivo de saída apropriado.

O filho exibe as informações finais na tela do terminal.

O processo também pode obter e herdar dinamicamente recursos do pai.



Operações sobre Processos

Quando um processo cria um novo processo, existem duas possibilidades de execução:

Possibilidade 1

O pai continua a ser executado concorrentemente com seus filhos.

Possibilidade 2

O pai espera até que alguns de seus filhos ou todos eles sejam encerrados.



Operações sobre Processos

Existem duas possibilidades de espaço de endereçamento para o novo processo:

1. Duplicata do Pai

O processo-filho é uma cópia idêntica do processo-pai, compartilhando o mesmo programa e dados originais.

Exemplo: *A chamada de sistema* ***`fork()`*** *no Unix cria um clone do processo para execução concorrente.*

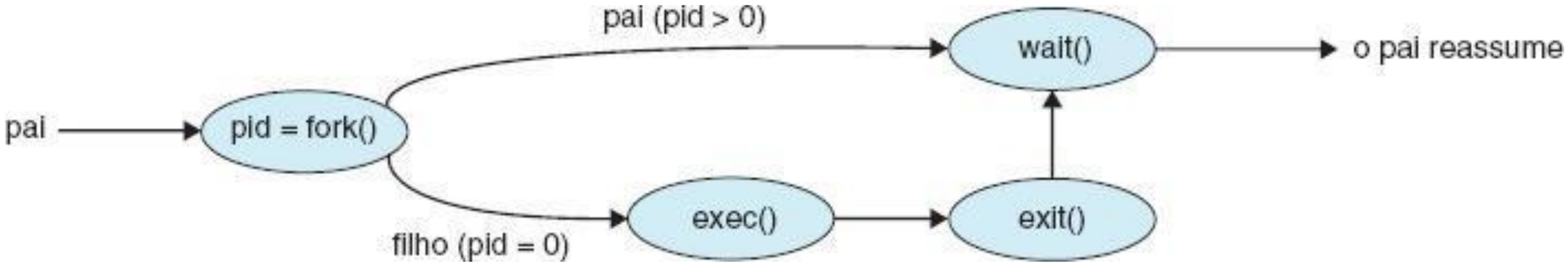
2. Novo Programa

O processo-filho carrega um novo programa na memória, passando a executar uma tarefa completamente diferente.

Exemplo: *A chamada* ***`exec()`*** *substitui o espaço de endereçamento por um novo arquivo executável.*



Operações sobre Processos





Encerramento de Processos

1. Fim da Execução

Um processo é encerrado quando termina a execução de seu último comando e solicita ao sistema operacional que o exclua, usando a chamada de sistema `exit()`.

2. Status para o Pai

Nesse momento, o processo pode retornar um valor de status (normalmente, um inteiro) para seu processo-pai por meio da chamada de sistema `wait()`.

3. Desalocação

Todos os recursos do processo – incluindo memória física e virtual, arquivos abertos e buffers de I/O – são totalmente liberados pelo sistema operacional.



Encerramento de Processos

Chamadas de Sistema Externas

O encerramento também pode ocorrer em outras circunstâncias. Um processo pode causar o encerramento de outro processo por meio de uma chamada de sistema apropriada por exemplo, `TerminateProcess()` no Windows.



Encerramento de Processos

Um pai pode encerrar a execução de um de seus filhos por várias razões, por exemplo:

01

Excesso de Recursos

O filho excedeu o uso de alguns dos recursos que recebeu. Para determinar se isso ocorreu, o pai deve ter um mecanismo para inspecionar o estado de seus filhos.

02

Tarefa Desnecessária

A tarefa atribuída ao filho não é mais requerida pelo sistema ou pela aplicação.

03

Encerramento Cascata

O pai está sendo encerrado e o sistema operacional não permite que um filho continue se seu pai for encerrado.



Encerramento de Processos

01

Desalocação

Quando um processo termina, seus recursos são imediatamente desalocados pelo sistema operacional.

02

Tabela de Processos

Sua entrada na tabela deve permanecer até que o pai chame wait ( ), pois ela contém o status de saída.

03

Processo Zumbi

Se o pai não chamou wait ( ), o processo vira "zumbi". Esse estado temporário ocorre com quase todos.



Encerramento de Processos

01

Processos Órfãos

Considere o que ocorreria se um pai não invocasse wait ( ) e, em vez disso, fosse encerrado deixando seus filhos órfãos.

02

Adoção pelo Init

O Linux e o UNIX resolvem esse problema designando o processo init como o novo pai dos processos órfãos.

03

Coleta Periódica

O processo init invoca wait ( ) periodicamente, coletando o status do órfão e liberando sua entrada na tabela.

Parte V - Comunicação Interprocessos



Comunicação Interprocessos

Razões para prover um ambiente que permita a cooperação entre processos:

01

Modularidade

Permite construir o sistema de forma modular, dividindo suas funções em processos ou threads separados.

Exemplo: Um player de vídeo que separa a decodificação do áudio, da renderização da imagem.

02

Conveniência

Permite que um usuário execute muitas tarefas ao mesmo tempo de maneira independente e paralela.

Exemplo: Ouvir música no Spotify enquanto digita um relatório e compila seu código.



Arquitetura Multiprocessos - Chrome

01

Conteúdo Rico

Muitos websites apresentam conteúdos ativos em JavaScript, Flash e HTML5 para fornecer uma experiência rica e dinâmica.

Exemplo: Jogos interativos rodando direto na página ou players de vídeo modernos.

02

Risco de Falhas

Essas aplicações web complexas podem conter bugs de software capazes de causar lentidão extrema ou até travar o navegador.

Exemplo: Um script mal otimizado que consome 100% da CPU e congela a tela do usuário.

03

Contexto Inicial

Isso não era um grande problema em navegadores antigos, pois eles eram projetados para exibir apenas um único website por vez.

Exemplo: Nos anos 90, abria-se uma janela totalmente nova para cada página que se desejava visitar.



Arquitetura Multiprocessos - Chrome

Navegação em Abas (Guias)

Permite que uma única instância do navegador abra vários websites ao mesmo tempo, de maneira totalmente independente.

Exemplo: Abrir o YouTube para ouvir música em uma aba enquanto pesquisa na Wikipédia em outra, sem que o travamento de uma aba afete a outra.

Isolamento de Processos

No Google Chrome, cada aba aberta funciona como um processo separado no sistema operacional.





Arquitetura Multiprocessos - Chrome

O Problema

Falha em Cadeia (Processo Único)

Se uma aplicação web em uma guia cair ou travar, o processo inteiro — incluindo todas as outras guias que estiverem exibindo outros sites — também cairá de forma simultânea.

Exemplo:

Um jogo travando em uma aba secundária faz com que você perca todo o texto que estava digitando em um formulário importante em outra aba.

A Solução

Arquitetura Multiprocessos (Chrome)

O Chrome resolve esse problema isolando os processos de forma independente. Ele identifica e gerencia três tipos principais:

- **Navegador**
- **Renderizadores**
- **Plug-ins**

Exemplo:

Se o player de vídeo de uma aba parar de responder, apenas aquela guia é fechada, mantendo todas as suas outras abas intactas e funcionando.



Arquitetura Multiprocessos - Chrome

Processo Navegador

Gerencia a interface do usuário, além de cuidar da entrada e saída (I/O) de disco e de rede. É **único** para toda a aplicação.

Exemplo:

Desenhar os botões de avançar/voltar, gerenciar a barra de endereços e controlar os downloads na máquina.

Processo Plug-in

Criado individualmente para cada tipo de plug-in em uso. Contém o código do componente e se comunica com os renderizadores.

Exemplo:

Executar de forma isolada extensões ou players legados (como Flash Player ou QuickTime) sem comprometer o resto do navegador.



Arquitetura Multiprocessos - Chrome

Processo Renderizador

Contém a lógica responsável por exibir e processar as páginas web. Ele manipula de forma direta elementos essenciais como HTML, JavaScript, CSS e imagens.

Exemplo:

Interpretar os códigos de um portal de notícias ou AVA para montar o layout visual e rodar as interações na tela.

Múltiplas Instâncias

Como regra geral, um novo processo renderizador independente é criado para cada website aberto em uma nova guia, permitindo que vários rodem ativos em paralelo.

Exemplo:

Ao abrir o YouTube e o Wikipedia em abas distintas, cada aba terá seu próprio processo renderizador isolado e em execução.



Arquitetura Multiprocessos - Chrome

A vantagem da abordagem multiprocessos é que os websites são executados isoladamente uns dos outros. Se um website falha, apenas o seu processo renderizador é afetado; todos os outros processos permanecem inalterados.

Além disso, processos renderizadores são executados em uma ~~**sandbox**~~ o que significa que o acesso ao I/O de disco e de rede é restrito, minimizando os efeitos de quaisquer invasões na segurança.



Arquitetura Multiprocessos - Chrome

Vantagens

Isolamento de Processos

A vantagem da abordagem multiprocessos é que os websites são executados isoladamente. Se um website falha, apenas o seu processo renderizador é afetado; todos os outros permanecem inalterados.

Exemplo:

Se uma aba do navegador travar devido a um script pesado, as outras abas abertas continuam funcionando normalmente.

Segurança via Sandbox

Os processos renderizadores são executados dentro de uma **sandbox**. Isso restringe o acesso direto ao disco rígido e à rede, contendo ameaças de forma eficaz.

Exemplo:

Se um site malicioso tentar roubar arquivos do seu computador ou instalar um vírus, a restrição da sandbox bloqueará essa tentativa.



Processos Cooperativos

O que são?

Processos cooperativos são aqueles que podem afetar ou ser afetados por outros processos no sistema.

Por isso, demandam um mecanismo de **Comunicação entre Processos (IPC)** que lhes permita trocar dados e informações de forma segura.

Exemplo prático:

Um navegador web que renderiza uma página de chat enquanto recebe novas mensagens em segundo plano de outro processo de rede.

Modelos de Comunicação (IPC)

Existem dois modelos fundamentais de comunicação entre processos:

1. Memória Compartilhada

Região comum de memória que ambos acessam diretamente para ler/escrever dados de forma rápida.

2. Transmissão de Mensagens

Troca de dados através de pacotes de mensagens intermediadas pelo sistema operacional (kernel).



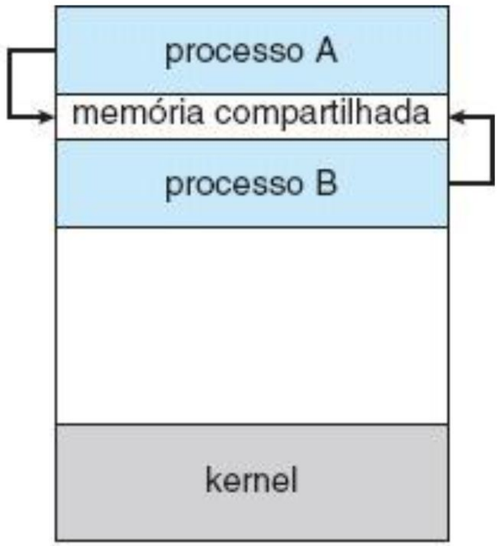
Processos Cooperativos

Memória Compartilhada

Estabelece-se uma região da memória que é compartilhada por processos cooperativos. Os processos podem, então, trocar informações lendo e gravando dados diretamente nessa região comum.

Exemplo prático:

Dois programas que compartilham uma tabela de dados na memória RAM para que um leia instantaneamente o que o outro acabou de escrever, sem precisar pedir ajuda ao sistema operacional.



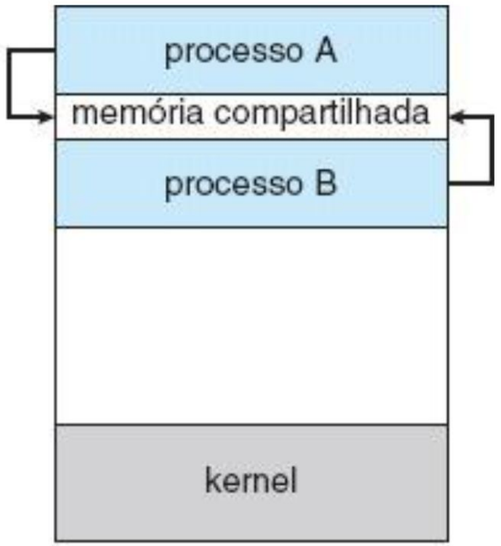


Processos Cooperativos

Memória Compartilhada

Pode ser **muito mais rápida** do que a transmissão de mensagens.

Isso ocorre porque evita a constante **intervenção do kernel** e chamadas de sistema, que tornam a comunicação por mensagens mais demorada.





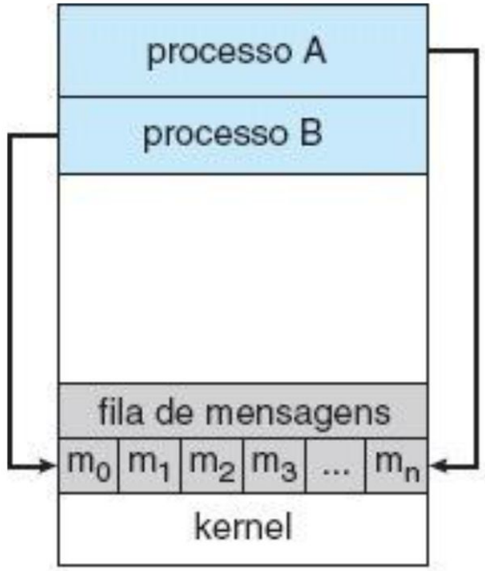
Processos Cooperativos

Transmissão de Mensagens

A comunicação ocorre por meio de mensagens trocadas diretamente entre os processos cooperativos, sem área compartilhada.

Exemplo prático:

Dois programas que se comunicam enviando dados a uma fila de mensagens gerenciada pelo sistema operacional (kernel), que garante a entrega de forma segura.





Processos Cooperativos

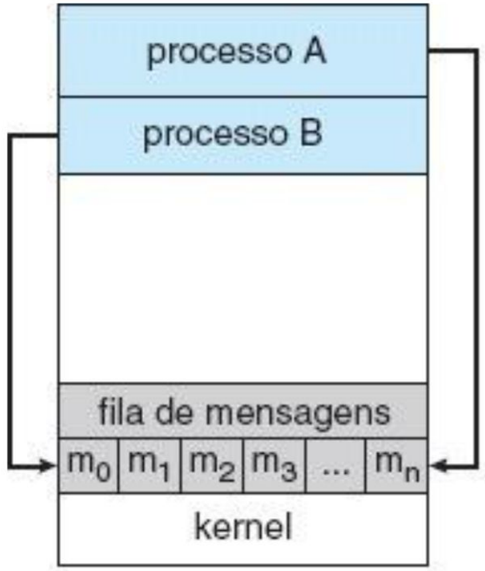
Transmissão de Mensagens

• É útil para a troca de pequenas quantidades de dados porque não é necessário evitar conflitos de acesso à memória.

• Mais fácil de ser implementada em um sistema distribuído do que a memória compartilhada.

Exemplo prático:

Serviços de chat (como WhatsApp ou Slack) ou APIs web onde pequenos pacotes de texto e dados são transmitidos de forma isolada e segura pelo sistema.





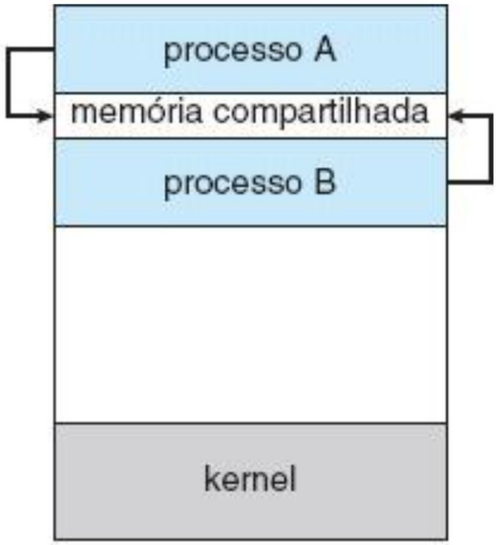
Processos Cooperativos

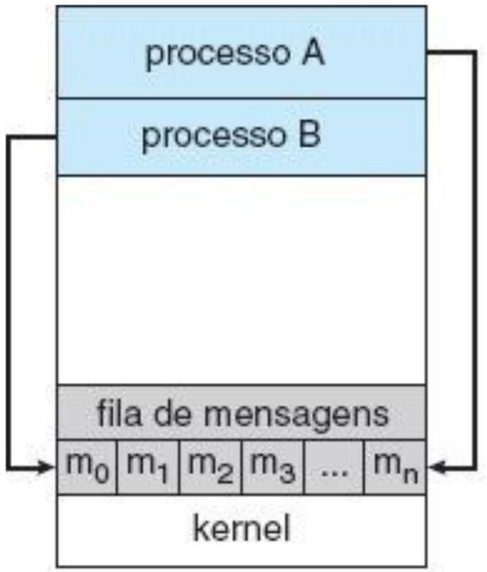
Modelos de Comunicação

Os dois modelos (memória compartilhada e transmissão de mensagens) são comuns e frequentemente coexistem nos sistemas operacionais modernos.

Desempenho & Tendência:

Em sistemas modernos de múltiplos núcleos (multi-core), pesquisas indicam que a **transmissão de mensagens** oferece melhor desempenho geral.







Sistemas de Memória Compartilhada

Conceito de Compartilhamento

A comunicação requer que os processos estabeleçam uma região de memória compartilhada para trocar informações lendo e gravando dados nela.

Controle do SO:

A forma e a locação dos dados são determinadas pelos próprios processos e não pelo sistema operacional.

Exemplo Prático

Produtor-Consumidor: Um processo escreve dados em um buffer comum e o outro lê esses dados concorrentemente.

Como em um **Servidor Web** enviando páginas HTML para o navegador do cliente de forma direta e veloz.



Sistemas de Memória Compartilhada

Produtor e Consumidor

Por exemplo, um servidor web **produz** (fornece) arquivos e imagens HTML que são **consumidos** (lidos) pelo navegador cliente.

Exemplo:

Navegador Web solicitando e renderizando uma página diretamente do servidor.

O Papel do Buffer

Para que funcionem concorrentemente, deve existir um **buffer**: uma área de armazenamento temporário preenchida pelo produtor e esvaziada pelo consumidor.

Exemplo:

Área de memória compartilhada para evitar que um processo trave enquanto o outro processa.



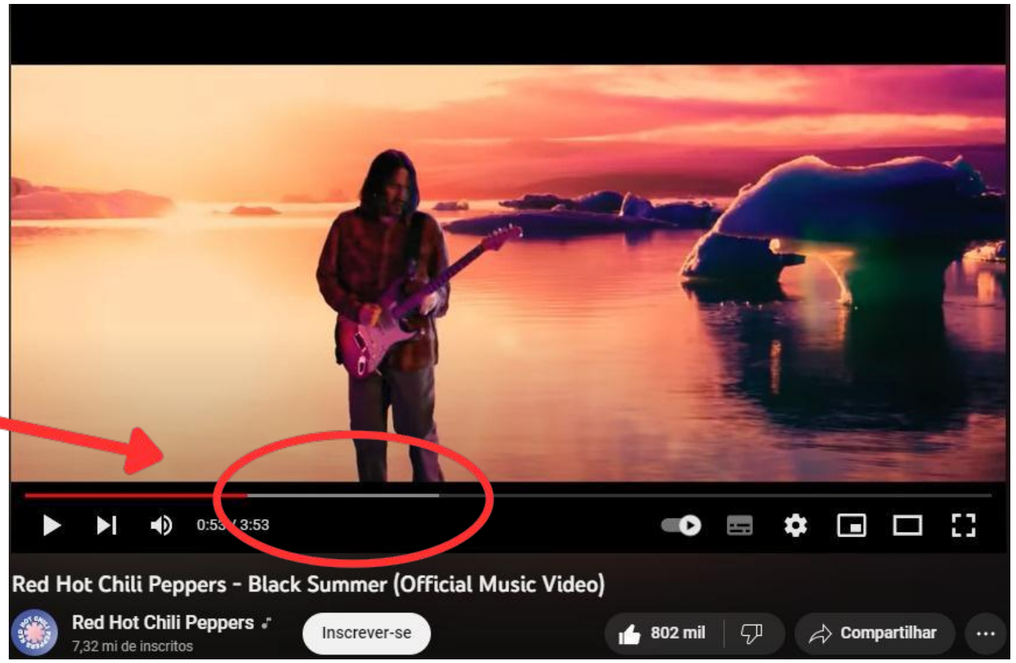
Sistemas de Memória Compartilhada

Exemplo Prático: YouTube

Ao assistir a um vídeo, a reprodução e o download ocorrem em paralelo por meio de um **buffer de memória compartilhado**.

Como funciona:

- **Produtor (Rede):** Baixa os dados do vídeo da internet e os escreve continuamente no buffer.
- **Consumidor (Player):** Lê os dados do buffer e exibe o vídeo de forma fluida para o usuário.





Sistemas de Transmissão de Mensagens

Como Funciona

O sistema operacional fornece um mecanismo para que os processos cooperativos se **comuniquem e sincronizem** suas ações de forma direta.

Diferencial:

Permite a cooperação **sem compartilhar** o mesmo espaço de endereçamento de memória.

Exemplo Prático

Pense em um **aplicativo de chat** ou em computadores diferentes trocando dados em uma rede local.

Aplicação Real:

Como os processos não dividem a mesma memória física, eles trocam pacotes explícitos de dados para cooperar a distância.



Sistemas de Transmissão de Mensagens

Ambiente Distribuído

Útil quando os processos em comunicação residem em **diferentes computadores** conectados por uma rede local ou Internet.

Exemplo Prático:

Um programa de chat onde os participantes se comunicam trocando mensagens entre seus dispositivos.

Operações Fundamentais

Para viabilizar a comunicação, o mecanismo de transmissão de mensagens fornece pelo menos **duas operações**:

- **send(message)** – Operação para enviar uma mensagem para o processo receptor.
- **receive(message)** – Operação para receber e processar a mensagem enviada.



Sistemas de Transmissão de Mensagens

O Link de Comunicação

Se os processos **P e Q** querem se comunicar, eles devem enviar e receber mensagens; para isso, um **link deve existir** entre eles.

Exemplo Prático:

Pense nisso como estabelecer uma linha telefônica ativa entre duas pessoas antes que elas possam conversar.

Modelos de Implementação

Há vários métodos para implementar logicamente o link e as operações **send() / receive()**:

- **Direta ou Indireta**: Comunicação com ou sem intermediários.
- **Síncrona ou Assíncrona**: Bloqueante ou não bloqueante.
- **Buffer Automático / Explícito**: Fila de mensagens temporária.



Sistemas de Transmissão de Mensagens

Comunicação Direta

Cada processo que deseja se comunicar deve **nomear explicitamente** o receptor ou o emissor da comunicação.

Exemplo Prático:

Como uma carta onde você deve obrigatoriamente preencher o destinatário correto antes de enviar.

Primitivas de Envio/Recepção

As operações básicas exigem a identificação clara de quem envia e de quem recebe:

- **send(P, message)** – Envia uma mensagem diretamente ao processo P.
- **receive(Q, message)** – Recebe uma mensagem vinda do processo Q.

Sistemas de Transmissão de Mensagens

Comunicação Indireta

As mensagens são enviadas e recebidas por meio de **caixas postais** (ou ports), que funcionam como intermediários onde as mensagens podem ser depositadas e removidas.

Exemplo Prático:

Pense em um sistema de caixas de correio de um condomínio, onde as pessoas depositam cartas em compartimentos específicos em vez de entregá-las em mãos.

Primitivas Indiretas

As operações de comunicação referenciam diretamente a caixa postal (A), permitindo o compartilhamento do canal:

send(A, message) – Envia uma mensagem para a caixa postal A.

receive(A, message) – Recebe uma mensagem a partir da caixa postal A.



Sistemas de Transmissão de Mensagens

Caixa Compartilhada

Um link só é estabelecido se ambos os processos possuírem uma **caixa postal compartilhada**.

Exemplo Prático:

Como duas pessoas que só conseguem trocar correspondência se tiverem acesso à mesma caixa física.

Múltiplos Processos

Um único link de comunicação pode estar associado a **mais de dois processos** ao mesmo tempo.

Exemplo Prático:

Um grupo de mensagens onde múltiplos membros podem ler e enviar informações no mesmo canal.

Vários Links

Entre cada par de processos, podem existir **vários links diferentes**, cada um ligado a uma caixa postal.

Exemplo Prático:

Duas pessoas que conversam ao mesmo tempo por e-mail, aplicativo de chat e rede de trabalho.



Sistemas de Transmissão de Mensagens

Comunicação Indireta

Agora, suponha que os processos **P1, P2 e P3** compartilhem a **caixa postal A**. O processo P1 envia uma mensagem para A, enquanto P2 e P3 executam um receive() a partir de A.

Que processo receberá a mensagem enviada por P1?

A resposta é: Depende!



Sistemas de Transmissão de Mensagens

Comunicação Indireta

Soluções para o problema ("Depende!"):

- **Opção 1:** Permitir que um link seja associado a, no máximo, dois processos.
- **Opção 2:** Permitir que, no máximo, um processo por vez execute uma operação receive().
- **Opção 3:** Permitir que o sistema selecione arbitrariamente o processo que receberá a mensagem, um por vez. O sistema pode definir um algoritmo para a seleção (como **round robin**, onde os processos se revezam).



Sistemas de Transmissão de Mensagens

Comunicação Indireta

Caixas Postais: Podem ser de propriedade do S.O ou do próprio processo.

Se pertencer ao processo:

- **Recepção Restrita:** O processo proprietário só recebe mensagens por esta caixa.
- **Envio Restrito:** O processo usuário só pode mandar mensagens para a caixa.
- **Encerramento:** Se o processo é encerrado, a caixa desaparece.
- **Notificação:** Os processos que enviam mensagens a esta caixa devem ser notificados.



Sistemas de Transmissão de Mensagens

Comunicação Indireta

Caixas Postais: Podem ser de propriedade do S.O ou do próprio processo.

Se pertencer ao S.O:

- A caixa independe e não está associada a nenhum processo específico. Se um processo vinculado a esta caixa for encerrado, ela não será encerrada e pode ser utilizada por outros processos.



Sistemas de Transmissão de Mensagens

Sincronização

Dinâmica da Transmissão

A troca de mensagens pode ocorrer de duas formas essenciais: com bloqueio (**síncrona**) ou sem bloqueio (**assíncrona**).

O Fluxo com Bloqueio:

- **Emissor Bloqueado:** O processo que envia fica totalmente travado após o envio.
- **Desbloqueio:** Sua operação só retorna quando o receptor final ou a caixa postal destino retira a mensagem.



Exemplo Prático

Como em uma entrega registrada, o remetente espera a assinatura de recebimento antes de poder ir embora.



Sistemas de Transmissão de Mensagens

Sincronização

Mecanismos de Sincronização

• **Envio sem bloqueio:** O processo emissor envia a mensagem e retoma a operação imediatamente. **Exemplo:** Como enviar um e-mail e continuar trabalhando sem esperar a resposta.

• **Recebimento com bloqueio:** O receptor fica travado até que uma mensagem esteja disponível. **Exemplo:** Esperar uma encomenda na porta de casa sem poder fazer outra coisa.

• **Recebimento sem bloqueio:** O receptor tenta recuperar uma mensagem; recebe uma resposta válida ou nula imediatamente. **Exemplo:** Olhar a caixa de correio: se houver carta você pega, senão continua seu dia.



Sistemas de Transmissão de Mensagens

Armazenamento em Buffer

Independentemente de a comunicação ser direta ou indireta, as mensagens trocadas residem em uma **fila temporária**. Essas filas podem ser implementadas de três maneiras:

1. **Capacidade Zero:** Nenhuma mensagem pode esperar no canal. O emissor é bloqueado até o receptor receber.
2. **Capacidade Limitada:** Fila de tamanho *n*. Se cheia, o emissor é bloqueado até liberar espaço.
3. **Capacidade Ilimitada:** Fila infinita. O emissor nunca é bloqueado ao enviar mensagens.

💡 **Exemplo Prático**

Pense na capacidade como uma caixa de correio: se não há caixa (zero), você precisa entregar em mãos; se a caixa cabe 5 cartas (limitada), você só pode deixar novas se houver espaço; se o depósito é infinito (ilimitada), você sempre pode postar.

Sistemas de Transmissão de Mensagens

Dúvidas?